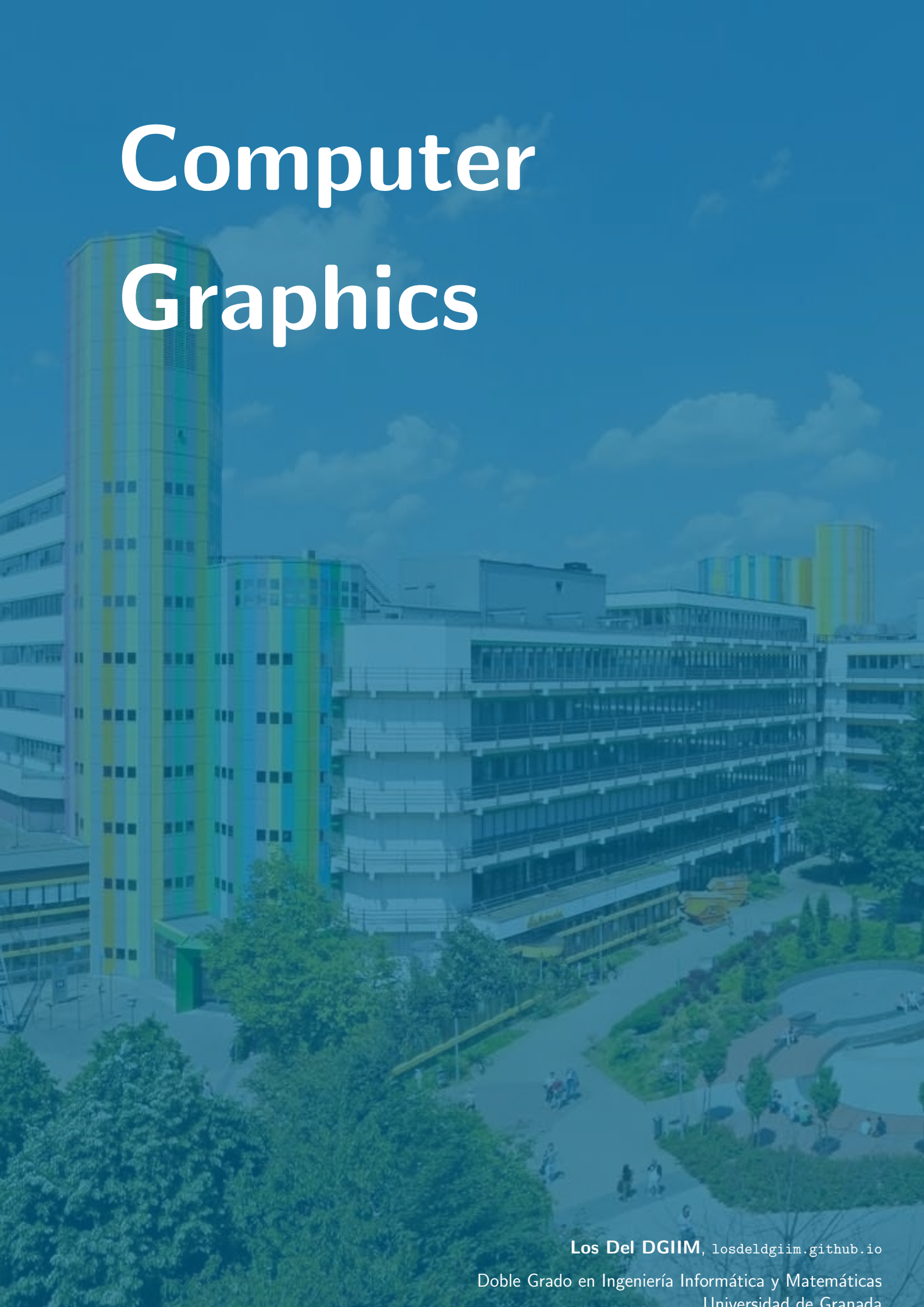
Computer Graphics



Los Del DGIIM, losdeldgiim.github.io

Doble Grado en Ingeniería Informática y Matemáticas

Universidad de Granada

Computer Graphics

Los Del DGIIM, losdeldgiim.github.io

Arturo Olivares Martos

Granada, 2026

Índice general

1. Ray and Path Tracing **7**
- 1.1. Ray Tracing . . . 8
  - 1.1.1. Diffuse surfaces . . . 9
- 1.2. Path Tracing . . . 9

2. Acceleration Structures **11**
- 2.1. Fewer rays . . . 11
- 2.2. Faster ray-object intersections . . . 11
  - 2.2.1. Barycentric interpolation . . . 11
  - 2.2.2. Ray-triangle intersection . . . 12
- 2.3. Fewer ray-object intersections . . . 13
  - 2.3.1. Bounding Volumes . . . 13
  - 2.3.2. Spatial subdivision . . . 15

3. Rasterization **19**
- 3.1. Fixed-Function Pipeline . . . 19
  - 3.1.1. Geometry . . . 19
  - 3.1.2. Geometry Processing . . . 20
  - 3.1.3. Rasterization . . . 25
  - 3.1.4. Fragment Processing . . . 26
- 3.2. Programmable Pipeline . . . 29

4. Color **31**
- 4.1. Technical Color Models . . . 31
  - 4.1.1. Additive Color Model: RGB . . . 32
  - 4.1.2. Subtractive Color Model: CMY . . . 32
- 4.2. Perceptual Color Models . . . 33
  - 4.2.1. HSV Color Model . . . 34
  - 4.2.2. HSL Color Model . . . 35
- 4.3. Other Color Models . . . 35
  - 4.3.1. YUV Color Model . . . 36
  - 4.3.2. XYZ Color Model . . . 36

5. Splines **41**
- 5.1. Polinomial Interpolation . . . 42
  - 5.1.1. Undetermined Coefficients . . . 42
  - 5.1.2. Lagrange Interpolation . . . 43
  - 5.1.3. Runge's Phenomenon . . . 43

5.2. Splines . . . 43
5.2.1. Constant Splines . . . 44
5.2.2. Linear Splines . . . 44
5.2.3. Hermite Cubic Splines . . . 46
5.2.4. Bezier Cubic Splines . . . 48
5.2.5. B-Spline Cubic Splines . . . 51
5.3. Parametric Surfaces . . . 55

6. Paths and Volumes **57**
6.1. The Rendering Equation . . . 57
6.1.1. Bidirectional Reflectance Distribution Function (BRDF) . . . 57
6.2. Participating Media . . . 58
6.2.1. Absorption . . . 58
6.2.2. Emission . . . 59
6.2.3. Scattering . . . 59
6.3. The Volumetric Rendering Equation . . . 60
6.4. Ray Casting Method . . . 60
6.4.1. Back-to-front compositing . . . 61
6.4.2. Front-to-back compositing . . . 61
6.5. Volumes Generation: Fractals . . . 62
6.5.1. Parameters and their effects . . . 63
6.5.2. Multifractals . . . 64

7. Radiosity **65**
7.1. The Radiosity Equation . . . 67
7.1.1. Diffuse Reflection Simplification . . . 68
7.1.2. Translating Radiance to Radiosity . . . 69
7.1.3. The Discretized Radiosity Equation . . . 69
7.2. The Radiosity Method . . . 71

8. Irradiance Volumes **73**
8.1. Generating the Irradiance Volume . . . 73
8.2. Interpolating the Irradiance Volume . . . 74

9. Photon Mapping **75**
9.1. Photon Emission Phase . . . 75
9.2. Rendering Phase . . . 77
9.3. Bias and Consistency . . . 79
9.4. Common Artifacts . . . 79
9.5. Progressive Photon Mapping . . . 79

10. Precomputed Radiance Transfer (PRT) **81**
10.1. Assumptions and Simplifications . . . 81
10.2. Converting the Integral into a Dot Product . . . 82
10.2.1. Spherical harmonics (SH) . . . 83
10.2.2. Monte Carlo Integration . . . 84
10.3. PRT Pipeline . . . 84
10.3.1. Precomputation Stage . . . 84

10.3.2. Rendering Stage 85

11. Ambient Occlusion **87**
11.1. Raytracing AO 88
11.2. Object Space AO 88
11.3. Bent Normals 89
11.4. Screen Space AO 90
11.4.1. Deferred Shading 90
11.4.2. SSAO Extensions 90

1. Ray and Path Tracing

In order to convert a 3D scene into a 2D image that can be displayed on a screen, there are two different approaches:

- Rasterization (Object-ordered rendering): It follows the logic: "Which pixels does this object cover?" It projects the geometry (usually triangles) onto the 2D image plane and fills the corresponding pixels. Its main advantage is its efficiency.

- Ray Tracing (Image-ordered rendering): It follows the logic: "What light reaches this pixel?" It shoots rays from the observer's eye through each pixel into the scene, calculating light bounces, reflections, and shadows. Its main advantage is its ability to produce highly realistic images, as it simulates the physical behavior of light more accurately.

During this chapter, we will cover the basics of ray tracing and path tracing, two important techniques in computer graphics for rendering images that follow the second approach. In order to introduce them, some previous concepts are required.

Definición 1.1 (Light Ray)**.** A *light ray* is a straight line along which light energy (photons) travels. It is defined by its origin and direction (which can also be given as a second point).

Definición 1.2 (Light Path)**.** A *light path* is a sequence of light rays that represent the journey of light from its source to the observer, including any interactions with objects in the scene (such as reflection, refraction, or absorption).

When a light ray interacts with an object, it can be absorved or its direction can be changed due to reflection or refraction. In order to render images, there are two main approaches:

- Going forward: rays are traced forward from the light source, so the entire scene would be perfectly illuminated. However, most of the rays will not reach the observer, so this method is inefficient.

- Going backward: rays are traced backward from the observer, so only the rays that reach the observer are considered. A ray is traced for determining the color of each pixel in the image.

Given the inefficiency of the forward approach, the backward approach is the one used in ray tracing and path tracing.

1.1. Ray Tracing

Ray tracing[1] is a rendering technique that simulates the way light interacts with objects in a scene to produce realistic images. For each pixel in the image, a *primary ray* is traced from the observer's eye through the pixel into the scene. Once the ray intersects with an object, three type of *secondary rays* can be generated:

- Shadow rays: For each light source, a shadow ray is traced from the intersection point to the light source to determine if the point is in shadow or not. If the shadow ray intersects another object before reaching the light source, the point is in shadow and does not receive direct illumination from that light source.

- Reflection rays: If the surface is reflective, a reflection ray is traced in the direction of the reflected ray, which is calculated based on the angle of incidence and the surface normal.

- Refraction rays: If the surface is translucid, a refraction ray is traced in the direction of the refracted ray, which is calculated based on Snell's law.

Therefore, for each intersection $N + 2$ rays are traced, where N is the number of light sources in the scene. The reflection and refraction rays are recursively considered as primary rays, so they can generate more secondary rays if they intersect with other objects. This process continues until once of the following conditions is met:

- A ray does not intersect with any object, in which case it contributes the background color to the pixel.

- A ray reaches a predefined maximum recursion depth, in which case it does not generate any more rays and contributes just its local illumination to the pixel.

The color of each pixel is determined by the color of its first intersection point. For each intersection point, the color is calculated as:

$$I = k_d I_{\text{direct}} + k_s I_{\text{reflection}} + k_t I_{\text{refraction}}$$

where $k_d, k_s, k_t \in \mathbb{R}_0^+$ such that $k_d + k_s + k_t = 1$ are three coefficients that determine the contribution of each type of ray to the final color, and I_{direct} is the color contribution from local illumination calculated using phong shading, $I_{\text{reflection}}$ is the color contribution from the intersection point of the reflection ray, and $I_{\text{refraction}}$ is the color contribution from the intersection point of the refraction ray.

Observación. As the reader may have noticed, the complexity of ray tracing increases exponentially, but the contribution of rays decreases exponentially, so in practice, a maximum recursion depth of 4 or 5 is usually sufficient for producing high-quality images.

[1] It is also kown as Whitted Ray Tracing, as Turner Whitted was one of its original developpers.

1.1.1. Diffuse surfaces

There are two main types of surfaces:

- Diffuse surfaces: they reflect light uniformly in all directions.
- Specular surfaces: they reflect light in a specific direction, following the law of reflection.

As the reader may notice, ray tracing just considers specular surfaces. In order to render diffuse surfaces, another reason to stop the recursion is included:

- A ray hits a diffuse surface, in which case it does not generate any more rays and contributes just its local illumination to the pixel.

This way, between the eye and the light source as many specular surfaces as we want can be included, but only one diffuse surface can be included just before the light source.

1.2. Path Tracing

Path tracing is also a rendering technique that simulates the way light interacts with objects in a scene to produce realistic images, similar to ray tracing. For each pixel in the image, N rays are traced from the observer's eye through the pixel into the scene, where N is a predefined number of samples per pixel. Once a ray intersects with an object, its direction is randomly changed according to the material properties of the surface, and a new ray is traced in that direction, generating therefore a path. This process continues until one of the following conditions is met:

- A ray does not intersect with any object, in which case it contributes the background color to the pixel.
- A ray reaches a predefined maximum depth, in which case it contributes just its local illumination to the pixel.

The color of each pixel is determined by the average color contribution of all the rays traced for that pixel. It should however be noted that "randomly" does not mean "uniformly". Depending on the material properties of the surface, from an specific direction, some directions will be more likely to be chosen than others.

- For perfect diffuse surfaces, the new direction is chosen uniformly in the hemisphere above the surface.
- For perfect specular surfaces, the new direction is the reflection direction.

2. Acceleration Structures

One of the main problems of ray tracing is that for each ray, we need to calculate its first intersection point with the objects in the scene. This is $O(mp)$, where m is the number of objects in the scene and p is the number of pixels in the image (as many rays as pixels are traced). If there are n polygons in total in the scene, the complexity of ray tracing is $O(np)$. In order to reduce this complexity, acceleration structures are used. In the following sections, we will cover the three most common acceleration methods.

2.1. Fewer rays

Two main techniques can be used for reducing the number of rays traced:

- Adaptive tree-depth control: The maximum recursion depth is not fixed, but it is determined adaptively based on the contribution of each ray to the final image. If a ray contributes very little to the final image, it is not traced further.
- Stochastic sampling: As seen in path tracing, only one secondary ray is traced for each intersection point, whose direction is randomly chosen according to the material properties of the surface.

However, there are some cases where these techniques are not effective. In these cases, other acceleration methods are required.

2.2. Faster ray-object intersections

In order to speed up the ray-object intersection tests, the concept of "barycentric interpolation" in a triangle can be used.

2.2.1. Barycentric interpolation

Barycentric interpolation is a method of interpolating values at the vertices of a triangle to find the value at any point inside the triangle. It uses the concept of barycentric coordinates, which are a set of three numbers that represent the relative position of a point with respect to the vertices of the triangle. The barycentric coordinates of a point P with respect to a triangle defined by vertices P_1, P_2, and P_3 are given by:

$$\lambda_1 = \frac{\Delta(P, P_2, P_3)}{\Delta(P_1, P_2, P_3)}, \quad \lambda_2 = \frac{\Delta(P, P_3, P_1)}{\Delta(P_1, P_2, P_3)}, \quad \lambda_3 = \frac{\Delta(P, P_1, P_2)}{\Delta(P_1, P_2, P_3)}$$

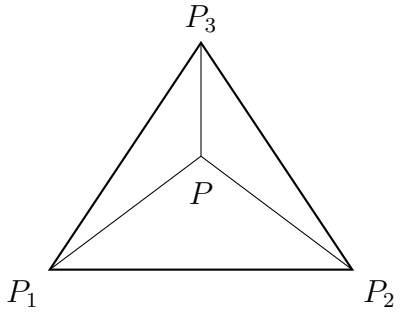


Figura 2.1: Barycentric coordinates of a point P with respect to a triangle defined by vertices P_1, P_2, and P_3.

where $\Delta(A, B, C)$ is the area of the triangle defined by points A, B, and C, as shown in Figure 2.1. If the point P is inside the triangle, then they satisfy the following properties:

- $\lambda_1 + \lambda_2 + \lambda_3 = 1$
- $0 \leqslant \lambda_i \leqslant 1$ for $i = 1, 2, 3$.

Once the barycentric coordinates of a point P are calculated, we can use them to interpolate any value defined at the vertices of the triangle. Given the values f_1, f_2, and f_3 at the vertices P_1, P_2, and P_3 respectively, the interpolated value at point P with barycentric coordinates λ_1, λ_2, and λ_3 is given by:

$$f(P) = \lambda_1 f_1 + \lambda_2 f_2 + \lambda_3 f_3$$

Two things should be noted about barycentric interpolation:

- It also interpolates the x and y coordinates of the point P in the triangle. These two conditions and the fact that $\lambda_1 + \lambda_2 + \lambda_3 = 1$ are sufficient to determine the barycentric coordinates of any point in the triangle.
- It can be proven that this interpolation is linear; and given that there is a unique linear interpolation for any set of values at the vertices of the triangle, it can be concluded that barycentric interpolation is just another way of performing linear interpolation.

2.2.2. Ray-triangle intersection

Any point P in the planed defined by a triangle defined by vertices P_1, P_2, and P_3 can be expressed as a linear combination of the vertices:

$$P = \lambda_1 P_1 + \lambda_2 P_2 + \lambda_3 P_3$$

where λ_1, λ_2, and λ_3 are the barycentric coordinates of P with respect to the triangle. In addition, any point P' on a ray defined by an origin P_S and a direction $\overrightarrow{D}$ can be expressed as:

$$P' = P_S + t\overrightarrow{D} \qquad t \in \mathbb{R}_0^+$$

Therefore, determining the intersection point of a ray with a triangle is equivalent to finding the values of λ_1, λ_2, λ_3, and t that satisfy the following system of equations:

$$\begin{cases} \lambda_1 P_1 + \lambda_2 P_2 + \lambda_3 P_3 = P_S + t\overrightarrow{D} \\ \lambda_1 + \lambda_2 + \lambda_3 = 1 \end{cases}$$

It should be noted that the first equation represents three equations, one for each coordinate. Therefore, we have a system of four equations with four unknowns. That system can be solved using Cramer's rule, but calculating the determinants can be computationally expensive. Instead, we can use the triple product of vectors to calculate the determinants more efficiently. After solving this system, we have to check if the solution is valid:

- $t \in \mathbb{R}_0^+$, as we are only interested in the intersection points that are in the direction of the ray (not behind the ray origin).
- $0 \leqslant \lambda_i \leqslant 1$ for $i = 1, 2, 3$, as we are only interested in the intersection points that are inside the triangle (not in the whole plane defined by the triangle).

This process is really fast, as:

- In order to determine λ_2, only one triple product of vectors is required. If λ_2 is not in the range $[0, 1]$, we can conclude that there is no intersection point between the ray and the triangle, and we can skip calculating λ_1 and λ_3.
- In order to determine λ_3, only a second triple product of vectors is required. If λ_3 is not in the range $[0, 1]$, we can conclude that there is no intersection point between the ray and the triangle.
- t is not even needed to be calculated, as the intersection point can be calculated using the barycentric coordinates.

2.3. Fewer ray-object intersections

There are two main techniques for reducing the number of ray-object intersection tests, which are developped in the next two sections.

2.3.1. Bounding Volumes

The main idea of bounding volumes is to enclose each object in the scene with a simple geometric shape (called a bounding volume) that can be tested for intersection with rays much faster than the original object (which may have thousands of polygons). There are two options:

- If the ray does not intersect the bounding volume, then it cannot intersect the original object, and we can skip the intersection test with the original object.
- If the ray intersects the bounding volume, then we need to perform the intersection test with the original object.

There are different types of bounding volumes, such as:

- Bounding spheres: The object is enclosed in a sphere defined by a center point and a radius.
  - Its intersection test with a ray is very fast, as it only requires solving a quadratic equation.
  - Its construction is also very fast, as it only requires calculating the center and the radius of the sphere that encloses the object. In addition, it doesn't have to be updated when the object is rigidly transformed.

  Despite these advantages, they also enclose a lot of empty space, which can lead to many false positives (i.e., rays that intersect the bounding sphere but do not intersect the original object).
- Axis-aligned bounding boxes (AABBs): The object is enclosed in a box whose edges are aligned with the coordinate axes.
  - Its intersection test with a ray is also fast.
  - Its construction is also very fast, as it only requires calculating the minimum and maximum coordinates of the object along each axis. However, it has to be updated when the object is rigidly transformed, as the minimum and maximum coordinates may change.
- Oriented bounding boxes (OBBs): The object is enclosed in a box that can be oriented in any direction.
  - Its intersection test with a ray is as fast as the AABB, but it can be more accurate, as it can fit the object better than the AABB.
  - Its construction is more expensive than the AABB, as it requires calculating the orientation of the box that best fits the object. However, it doesn't have to be reconstructed when the object is rigidly transformed, as the same transformation can be applied to the box.

However, using bounding volumes alone only reduces the number of ray-object intersection tests by a constant factor, as each object is still tested for intersection with the ray. The complexity of ray tracing with bounding volumes is still $O(np)$, but with a smaller constant factor. In order to achieve a better complexity, a hierarchy of bounding volumes can be used.

Bounding Volume Hierarchies (BVH)

A bounding volume hierarchy (BVH) is a tree structure where each node contains a bounding volume that encloses a subset of the objects contained in its father. The algorithm is therefore as follows:

- If the ray does not intersect the bounding volume of a node, then it cannot intersect any of the objects contained in that node, and we can skip the intersection tests with all those objects.

- If the ray intersects the bounding volume of a node, then we need to check the intersection with the bounding volumes of its children nodes, and so on until we reach the leaf nodes.

That way, we can reduce the number of ray-object intersection tests significantly, as many rays will not intersect the bounding volumes of the nodes in the hierarchy. The complexity of ray tracing with a BVH is significantly reduced to logarithmic.

2.3.2. Spatial subdivision

In this technique, instead of enclosing the objects in bounding volumes, the space is divided into disjoint regions (called cells), and each cell contains a list of the objects that are intersecting it. There are different types of spatial subdivision, such as the following.

Uniform grids

In this technique, the space is divided into a regular grid of cells, all of them of the same size. Traversing the grid is very fast, but it is a poor choice if the objects in the scene are not uniformly distributed, as many cells will be empty and others will contain many objects, leading to many ray-object intersection tests. Therefore, the following techniques are used to adapt the grid to the distribution of the objects in the scene.

Octrees

In this technique, the space is recursively subdivided into eight octants (hence the name octree). If one of the octants contains more than a certain number of objects, it is further subdivided into eight smaller octants, and so on until a certain maximum depth is reached or until all the octants contain a small number of objects. The result is a tree structure where each node represents an octant of the space, and each leaf node contains a octant that contains a small number of objects. This way, the grid is adapted to the distribution of the objects in the scene, leading to fewer ray-object intersection tests.

Each division is in fact three divisions:

- A division along the x-axis, with a plane perpendicular to the x-axis located at the middle of the cell along the x-axis, which divides the space in two equal parts along the x-axis.
- A division along the y-axis, with a plane perpendicular to the y-axis located at the middle of the cell along the y-axis, which divides the space in two equal parts along the y-axis.
- A division along the z-axis, with a plane perpendicular to the z-axis located at the middle of the cell along the z-axis, which divides the space in two equal parts along the z-axis.

KD trees

This idea follows the same idea as octrees, but instead of dividing the space into eight octants, each division uses just one plane.

- The plane direction is chosen to be perpendicular to one of the coordinate axes (i.e., x, y, or z). The order of the axes is chosen in a round-robin manner (i.e., x, then y, then z, then x again, and so on).
- There are however infinite possible positions for the plane along the chosen axis. In order to reduce the number of possible positions, only the planes located at the boundaries of the objects in the scene are considered. An optimization approach is used, selecting the plane that minimizes an specific cost function.

Binary Space Partitioning (BSP) trees

This idea follows the same idea as KD trees, but instead of restricting the plane direction to be perpendicular to one of the coordinate axes, any plane can be used for the division. Therefore, not only the direction but also the position of the plane can be optimized.

Detecting where a point P is located within the BSP tree is done by traversing the tree from the root node to the leaf nodes, at each node checking on which side of the plane the point P is located, and then moving to the corresponding child node until a leaf node is reached. In order to know on which side of the plane defined by a point P_0 and a normal vector $\overrightarrow{N}$ the point P is located, the following dot product can be calculated:

$$(P - P_0) \cdot \overrightarrow{N}$$

If the result is positive, then P is located on the side of the plane towards which the normal vector $\overrightarrow{N}$ is pointing. If the result is negative, then P is located on the opposite side of the plane. If the result is zero, then P is located on the plane itself.

Cost Function for KD and BSP trees

The cost function is used to determine the optimal position and direction of the dividing plane in both KD and BSP trees. An heuristic called *Surface Area Heuristic (SAH)* is commonly used for this purpose. It takes into account three components:

- The traversal cost, t_t, which is the cost of traversing the structure of the tree. It is usually constant, but it should be taken into account.
- The cost of processing each of the two parts of the space after the division. For each part, the cost is calculated taking three factors into account:
  - The probability of a ray intersecting that part of the space. As it depends on the surface area of that part of the space, it can be calculated as the ratio of the surface area of that part to the surface area of the whole space.

$$P_i = \frac{S_i}{S}$$

- The number of primitives in that part of the space, N_i. If one of the parts contains a lot of primitives, it should have a smaller probability of being intersected by a ray, as it will lead to more ray-object intersection tests.
- The cost of intersecting a ray with the primitives in that part of the space, t_p.

Therefore, the cost of processing each part of the space can be calculated as:

$$C_i = P_i \cdot N_i \cdot t_p$$

Therefore, the total cost of a division can be calculated as:

$$C = t_t + C_1 + C_2 = t_t + P_1 \cdot N_1 \cdot t_p + P_2 \cdot N_2 \cdot t_p$$

3. Rasterization

As we introduced in the first chapter, there are two main approaches for rendering images in computer graphics: rasterization and ray tracing. During this chapter, we will discuss the rasterization graphics pipeline. It starts from the initial creation of 3D models and ends with the final display of the rendered image. There are two types of graphics pipelines: the fixed-function pipeline and the programmable pipeline.

3.1. Fixed-Function Pipeline

The fixed-function pipeline is a traditional graphics pipeline that was widely used in older graphics hardware. It consists of a series of predefined stages that perform specific tasks. Each stage has a fixed function, and the data flows through these stages in a specific order. During the following subsections, we will discuss the stages of the fixed-function pipeline in detail.

3.1.1. Geometry

During this stage, the 3D models are created and defined. This includes defining the vertices, normals, texture coordinates, and other attributes of the 3D objects. In order to create the 3D models, the following primitives are used:

- `GL_POINTS`
- `GL_LINES`: A series of disconnected straight lines.
- `GL_LINE_STRIP`: A series of connected lines.
- `GL_LINE_LOOP`: A series of connected lines that form a loop.
- `GL_POLYGON`: A filled polygon defined by a series of vertices.
- `GL_TRIANGLES`: A series of disconnected triangles.
- `GL_TRIANGLE_STRIP`: A series of connected triangles. The first three vertices define the first triangle, and each subsequent vertex forms a new triangle with the previous two vertices.
- `GL_TRIANGLE_FAN`: A series of connected triangles that share a common central vertex. The first vertex is the center of the fan, and each subsequent vertex forms a new triangle with the center vertex and the previous vertex.
- `GL_QUADS`: A series of disconnected quadrilaterals.

- `GL_QUAD_STRIP`: A series of connected quadrilaterals. The first four vertices define the first quadrilateral, and each subsequent pair of vertices forms a new quadrilateral with the previous two vertices.

Even though all of these primitives are supported by OpenGL, all of the primitives will be converted to triangles, as they are the most optimized one in modern graphics cards. In order to specify the vertices of the primitives, a process evolved from immediate mode to vertex arrays and eventually to vertex buffer objects (VBOs).

1. Immediate Mode: In this mode, vertices are specified one at a time. It is simple to use but inefficient, as it requires multiple function calls for each vertex. The geometry is then stored in the code.

2. Vertex Arrays: This mode allows you to specify an array of vertices and their attributes, which can be more efficient than immediate mode. The geometry is then stored in the RAM.

3. Vertex Buffer Objects (VBOs): This mode allows you to store vertex data in the GPU's memory, which can significantly improve performance.

3.1.2. Geometry Processing

During this stage, the vertices defined in the geometry stage are processed to prepare them for rendering. There are different spaces in which the vertices are processed:

- Object Space: The original space in which the vertices are defined.

- World Space: The space in which the vertices are transformed to position them in the scene.

- Camera/Eye Space: The space in which the vertices are transformed to position them relative to the camera (the camera is at the origin).

- Clip Space: The space in which the vertices are transformed to prepare them for clipping (removing parts of the geometry that are outside the view frustum).

- Normalized Device Space: The space in which the vertices are transformed to fit within a standardized coordinate system (usually a cube from -1 to 1 in all axes).

- Screen Space: The final space in which the vertices are transformed to fit the actual screen dimensions.

In order to transform the vertices from one space to another, several transformations are applied.

Model Transformation

The model transformation is the transformation that takes the vertices from object space to world space. It is used to position the objects in the scene. It is usually represented as a combination of different transformations, and mathematically it can be represented as a single matrix M_{model} that is the product of the individual transformation matrices (taking the order of transformations into account).

$$P_{\text{world}} = M_{\text{model}} \cdot P_{\text{object}}$$

When a transformation M is applied to a vertex P, the normal vector N of the vertex is also transformed. Someone might think that the normal vector can be transformed using the same transformation M, but this is not always the case. The normal vector should be transformed using the inverse transpose of the transformation matrix M to ensure that it remains perpendicular to the surface after the transformation. Mathematically, the transformed normal vector N' can be calculated as follows:

$$N' = (M^{-1})^t \cdot N$$

Demostración. To understand why the normal vector should be transformed using the inverse transpose of the transformation matrix, we need to consider how transformations affect the geometry of the surface. Let n be the normal vector of a surface at a point P and t be the tangent vector at the same point. For clarity, let's use T_O to denote the transformation applied to the points (the tangent) and T_N to denote the transformation applied to the normal vector. We want to ensure that after applying the transformations, the normal vector remains perpendicular to the tangent vector, which means:

$$(T_N \cdot n) \perp (T_O \cdot t) \iff (T_N \cdot n) \cdot (T_O \cdot t) = 0 \iff (T_N \cdot n)^t \cdot (T_O \cdot t) = 0 \iff$$
$$\iff n^t \cdot T_N^t \cdot T_O \cdot t = 0$$

Given that n is perpendicular to t, we have $n^t \cdot t = 0$. Therefore, for the transformed normal vector to remain perpendicular to the transformed tangent vector, we need:

$$T_N^t \cdot T_O = I \iff T_N^t = T_O^{-1} \iff T_N = (T_O^{-1})^t$$

□

This should always be taken into account when applying transforming normals, *the normal vector should be transformed using the inverse transpose of the transformation matrix applied to the points.*

View Transformation

The view transformation is the transformation that takes the vertices from world space to camera/eye space. It is used to position the camera in the scene and to define the view direction. In OpenGL, the camera is always positioned at the origin of the coordinate system, looking in the z direction. As the camera can't move, the view transformation is achieved by applying the inverse of the camera's transformation to the vertices. Let's say that we want to position the camera at a point C in world space, looking at a direction D, with the up vector U'. We can define the view transformation using the following steps:

1. First of all, as we want to position the camera at point C, we need to apply a translation transformation that moves the world in the opposite direction of C. This can be represented using a translation matrix T that translates by $-C$.

2. Next, we need to align the camera's view direction with the z axis. To do this, we only know the direction D that the camera is looking at.

   a) We can calculate the right vector R of the camera using the cross product of the view direction D and the up vector U'. This can be represented as $R = D \times U'$.

   b) Then, we can calculate the up vector U of the camera using the cross product of the right vector R and the view direction D. This can be represented as $U = R \times D$. We need to do this step because the original up vector U' may not be perpendicular to the view direction D, so we need to calculate a new up vector U that is perpendicular to both R and D.

   This way, the matrix $[R, U, D]$ maps this orthogonal basis to the standard basis of the camera space. Therefore, the inverse of this transformation, which aligns the camera space with the standard basis, is:

$$[R, U, D]^{-1}$$

Therefore, the view transformation can be represented as the product of the translation and the inverse of the rotation:

$$M_{\text{view}} = [R, U, D]^{-1} \cdot T$$

The order of the transformations is important, as the translation should be applied before the rotation to ensure that the camera is correctly positioned and oriented in the scene.

Lighting and Shading

Before moving to the next stage, lighting and shading should be considered, because they are the processes that determine the color and intensity of the pixels in the final rendered image. Lighting is the process of calculating how light interacts with the surfaces of the objects in the scene, while shading is the process of determining the color of each pixel based on the lighting calculations. The Phong Lighting Model is a widely used model for calculating the lighting in computer graphics. It uses the following notation.

Notación. The notation used in the Phong Lighting Model is shwon in Figure 3.1.

- X is the point on the surface being shaded.
- N is the normal unitary vector at point X.
- L is the unitary vector from point X to the light source.

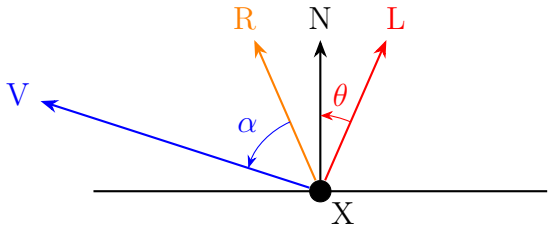


Figura 3.1: Notation used in the Phong Lighting Model.

- V is the unitary vector from point X to the viewer (camera).
- R is the reflection of the light vector L around the normal vector N, calculated as $R = 2(N \cdot L)N - L$.

  This calculation is valid because $N \cdot L = \cos(\theta)$ gives the projection of L onto N (as the cosine is the adjacent side of the triangle formed by L and N, and $\|L\| = 1$).

The Phong Lighting Model calculates the color of a pixel using three components: ambient, diffuse, and specular.

- Ambient Component: This component represents the constant background light in the scene. It is calculated as:

  $$\text{Ambient}(X) = C_{X,a} \cdot C_{L,a}$$

  where $C_{X,a}$ is the ambient color of the surface at point X and $C_{L,a}$ is the ambient color of the light source.

- Diffuse Component: This component represents the light that is scattered in all directions when it hits a surface. It is calculated as:

  $$\text{Diffuse}(L, X) = C_{X,d} \cdot C_{L,d} \cdot \text{máx}(0, \cos(\theta)) = C_{X,d} \cdot C_{L,d} \cdot \text{máx}(0, N \cdot L)$$

  where $C_{X,d}$ is the diffuse color of the surface at point X, $C_{L,d}$ is the diffuse color of the light source, and θ is the angle between the normal vector N and the light vector L.

- Specular Component: This component represents the light that is reflected in a specific direction when it hits a surface. It is calculated as:

  $$\text{Specular}(L, X, V) = C_{X,s} \cdot C_{L,s} \cdot \text{máx}(0, \cos(\alpha))^n = C_{X,s} \cdot C_{L,s} \cdot \text{máx}(0, R \cdot V)^n$$

  where $C_{X,s}$ is the specular color of the surface at point X, $C_{L,s}$ is the specular color of the light source, α is the angle between the reflection vector R and the view vector V, and n is the shininess coefficient that controls the size of the specular highlight.

The final color of the pixel is then calculated as the sum of these three components:

$$\text{Color}(L, X, V) = \text{Ambient}(X) + \text{Diffuse}(L, X) + \text{Specular}(L, X, V)$$

Perspective Transform

In this stage, the vertices are transformed from camera/eye space to clip space. The next definition is important.

Definición 3.1 (Frustrum)**.** The view frustum is a geometric shape that represents the volume of space that is visible to the camera. It can be defined in several ways (using the field of view, just the planes...). Two important planes are:

- Near Plane: The plane that is closest to the camera. Objects that are closer than this plane will not be rendered.
- Far Plane: The plane that is farthest from the camera. Objects that are farther than this plane will not be rendered.

Two type of frustrums can be defined:

- Perspective Frustum: A frustum that represents a perspective projection, where objects that are farther from the camera appear smaller. This type of frustum looks like a truncated pyramid, with the near plane being smaller than the far plane.
- Orthographic Frustum: A frustum that represents an orthographic projection, where objects that are farther from the camera appear the same size as objects that are closer to the camera. This type of frustum looks like a rectangular box, with the near and far planes being the same size. This type of projection is often used for 2D rendering or for technical drawings where accurate measurements are important. It represents that the viewer is at the infinity.

As already explained, the perspective transformation is used to transform the vertices from camera/eye space to clip space. It is represented as a matrix that prepares the vertices for perspective division. The exact form of the perspective transformation matrix depends on the parameters of the view frustum (how was it given), but it generally includes terms that account for the field of view, aspect ratio, and the near and far planes. We will simply call it $M_{\text{projection}}$, without going into the details of its construction. Therefore, what the programmer should implement in OpenGL is:

$$P_{\text{clip}} = M_{\text{projection}} \cdot M_{\text{view}} \cdot M_{\text{model}} \cdot P_{\text{object}}$$

Clipping

The clipping stage removes any vertices that are outside the view frustum. Here, the vertices are transformed from clip space to normalized device space by performing perspective division. Given that the vertices are in homogeneous coordinates and we need the last w coordinate to be 1, the other coordinates are divided by w to achieve this. After perspective division, and given that the correct perspective transformation was applied, the vertices will be in normalized device space, and only the vertices that are within the range of $[-1, 1]$ in all axes will be kept for further processing. The vertices that are outside this range will be clipped, meaning that they will be discarded and not rendered.

Viewport Transformation

In this last stage, the vertices are transformed from normalized device space to screen space. This transformation maps the normalized device coordinates to the actual pixel coordinates on the screen. The viewport transformation is defined by the viewport parameters, which specify the position and size of the viewport on the screen. The viewport transformation can be represented as a matrix that scales and translates the vertices to fit within the specified viewport. After this transformation, the vertices are ready to be rasterized and rendered on the screen.

3.1.3. Rasterization

The rasterization stage is the process of converting the vertices and primitives into pixels on the screen. During this stage, the graphics pipeline determines which pixels on the screen correspond to each primitive, and a fragment is generated for each of these pixels. The data for each vertex (color, texture coordinates, etc.) is interpolated across the primitive to determine the attributes of each fragment.

Data Interpolation

Given $n+1$ points $\overrightarrow{x}_i \in \mathbb{R}^k$ with their corresponding values $f_i \in \mathbb{R}$, $i \in \{0, \ldots, n\}$, interpolating is finding a function $f : \mathbb{R}^k \to \mathbb{R}$ such that $f(\overrightarrow{x}_i) = f_i$ for all $i \in \{0, \ldots, n\}$. When achieved, the function f can be used to estimate the value of f at any point $\overrightarrow{x} \in \mathbb{R}^k$ by evaluating $f(\overrightarrow{x})$.

Linear interpolation is a simple method of interpolation that only considers the linear functions f, which can be represented as $f(\overrightarrow{x}) = \overrightarrow{a} \cdot \overrightarrow{x} + b$ for some $\overrightarrow{a} \in \mathbb{R}^k$ and $b \in \mathbb{R}$. Given that we have $n + 1$ constraints (one for each point) and $k + 1$ unknowns (the components of $\overrightarrow{a}$ and b), we can solve for the coefficients of the linear function f if $n + 1 \leqslant k + 1$. In our case, we are interested in interpolating the attributes of the vertices across the primitives, which means that we have 3 vertices (for triangles) and two dimensions (the screen coordinates), so $k = 2$. Therefore, our aim is to find $a, b, c \in \mathbb{R}$ such that $f(x, y) = ax + by + c$ and $f(\overrightarrow{x}_i) = f_i$ for $i \in \{0, 1, 2\}$. This can be achieved by solving the system of equations of Equation (3.1).

$$\begin{cases} ax_0 + by_0 + c = f_0 \\ ax_1 + by_1 + c = f_1 \\ ax_2 + by_2 + c = f_2 \end{cases} \implies \begin{pmatrix} x_0 & y_0 & 1 \\ x_1 & y_1 & 1 \\ x_2 & y_2 & 1 \end{pmatrix} \begin{pmatrix} a \\ b \\ c \end{pmatrix} = \begin{pmatrix} f_0 \\ f_1 \\ f_2 \end{pmatrix} \tag{3.1}$$

However, this does not take the perspective into account. To achieve perspective-correct interpolation, the depth value of each vertex is interpolated in a way that accounts for the perspective distortion; and then the attributes are interpolated using the interpolated depth values.

Shading

Shading is the process of determining the color of each pixel based on the lighting calculations. During rasterization, the attributes of the vertices are interpolated across the primitive to determine the attributes of each fragment. There are different

ways to apply the Phong Lighting Mode, depending on what is interpolated and when is the color calculated:

- Flat Shading: The color is calculated for each primitive (e.g., triangle) using a single value of each attribute for the whole primitive (usually the corresponding values of the first vertex). This results in a faceted appearance, as each primitive has a single color.
- Gouraud Shading: The color is calculated at each vertex , and then interpolated across the primitive. This results in a smoother appearance than flat shading, but it can produce artifacts such as Mach bands.
- Phong Shading: The normal vector is interpolated across the primitive, and the color is calculated for each fragment using the interpolated normal vector. This results in a smoother appearance than Gouraud shading and can produce more accurate lighting effects.

3.1.4. Fragment Processing

The fragment processing stage is almost the final stage of the graphics pipeline, where the fragments generated during rasterization are processed to determine their final color and whether they should be rendered on the screen. During this stage, several operations can be performed on the fragments, such as depth testing, stencil testing, blending, and more. The specific operations performed during fragment processing depend on the settings and configurations of the graphics pipeline.

Textures

A texture map is a discretely sampled multi-dimensional function containing material properties, like color or normals. Texture mapping is the process of applying a texture map to a 3D model to add detail and realism to the rendered image. During fragment processing, the texture coordinates of each fragment are used to sample the corresponding attribute from the texture map, which can then be used. Some concepts about textures are:

- Wrapping: Texture coordinates are usually planned to be in the range of $[0, 1]$, but sometimes they can be outside this range. Wrapping is the process of determining how to handle texture coordinates that are outside the $[0, 1]$ range. Common wrapping modes include:
  - `GL_REPEAT`: The texture is repeated when the texture coordinates are outside the $[0, 1]$ range.
  - `GL_MIRRORED_REPEAT`: The texture is repeated and mirrored when the texture coordinates are outside the $[0, 1]$ range.
  - `GL_CLAMP_TO_EDGE`: The texture coordinates are clamped to the edge of the texture when they are outside the $[0, 1]$ range. This means that the texture will be stretched to fill the area outside the $[0, 1]$ range.

  One mode should be selected for each texture coordinate (e.g., s and t).

- Filtering: Usually, the texture maps are of a different resolution than the rendered image, which means that the texture needs to be filtered to determine the color of each pixel. There are two types of filtering:
  - Minification: The process of determining the color of a fragment when the texels (texture pixels) are smaller than the screen pixels.
  - Magnification: The process of determining the color of a fragment when the texels are larger than the screen pixels.

  Common filtering modes include:
  - `GL_NEAREST`: The color of the fragment is determined by the color of the nearest texel. In minification, this can lead to aliasing artifacts, while in magnification, it can lead to a blocky appearance.
  - `GL_LINEAR`: The color of the fragment is determined by the bilinear interpolation of the colors of the 4 nearest texels. In minification it does not make a big difference (considering 4 texels when the pixel may be covered by 100 texels does not improve the quality), but in magnification it can improve the appearance of the texture, making it appear smoother.
- MIP Mapping: MIP mapping is a technique used to improve the quality of textures when they are minificated. It involves creating multiple levels of detail for a texture, where each level is a downscaled version of the original texture. During rendering, the appropriate level of detail is selected based on the distance and angle of the fragment being rendered. This can help to reduce aliasing artifacts and improve the overall appearance of the texture, as no minification is applied when the appropriate level of detail is selected.

Fragment Tests

After the fragment processing stage, several tests can be performed on the fragments to determine whether they should be rendered on the screen or discarded. Before performing these tests, the content of each pixel should be explained. Each pixel and fragment on the screen has a buffer that stores:

- Color: The color of the pixel, usually represented as RGB (red, green, blue).
- Alpha: The alpha value of the pixel, which people tend to use it to represent the transparency of the pixel. However, it can be used for other purposes as well.
- Depth: The depth value of the current pixel, which is the depth value of the fragment that is currently rendered at that pixel (the closest fragment to the camera). It is stored in the depth buffer.
- Stencil: The stencil value of the pixel, which is used for stencil testing. The programmer can define the stencil value for each pixel and store it in the stencil buffer.

Once this has been explained, some common fragment tests include:

- Alpha Test: This test discards fragments based on their alpha value. It consists of a comparison between the alpha value of the *fragment* and a *reference value*, using a specified comparison function (e.g., less than, greater than, equal to). If the comparison fails, the fragment is discarded and not rendered on the screen.
- Depth Test: This test discards fragments based on their depth value. It compares the depth value of the *fragment* with the depth value of the corresponding *pixel* in the depth buffer. Different comparison functions can be used (e.g., less than, greater than, equal to) to determine whether the fragment should be discarded or rendered. However, the most common comparison function is "less than", (`GL_LESS`), which means that a fragment will only be rendered if it is closer to the camera than the existing pixel. If that is the case, it passes the depth test and is rendered on the screen, and the depth buffer is updated with the new depth value. If the fragment is farther from the camera than the existing pixel, it fails the depth test and is discarded.
- Stencil Test: This test discards fragments based on their stencil value. It compares the stencil value of the *pixel* where the fragment would be rendered with a *reference value*, using a specified comparison function. If the comparison fails, the fragment is discarded and not rendered on the screen. The stencil test can be used for various effects, such as creating shadows, outlining objects, or implementing complex masking operations.

Blending

Blending is the process of combining the color of a fragment with the color of the corresponding pixel in the framebuffer to produce a final color that is rendered on the screen. Blending is typically (but not only) used to achieve transparency effects, where the color of the fragment is blended with the color of the background to create a semi-transparent appearance. The blending operation is defined by:

$$C_{\text{final}} = C_{\text{source}} \cdot F_{\text{source}} \odot C_{\text{destination}} \cdot F_{\text{destination}}$$

where:

- C_{final} is the final color that is rendered on the screen.
- C_{source} is the color of the fragment being processed.
- $C_{\text{destination}}$ is the color of the corresponding pixel in the framebuffer.
- F_{source} and $F_{\text{destination}}$ are the blending factors for the source and destination colors, respectively. These factors determine how much of each color contributes to the final color. Common blending factors include:
  - `GL_ZERO`: The factor is 0, meaning that the corresponding color does not contribute to the final color.
  - `GL_ONE`: The factor is 1, meaning that the corresponding color fully contributes to the final color.

  - `GL_SRC_ALPHA`: The factor is equal to the alpha value of the source color, meaning that the contribution of the source color is proportional to its transparency.
  - `GL_ONE_MINUS_SRC_ALPHA`: The factor is equal to 1 minus the alpha value of the source color, meaning that the contribution of the destination color is proportional to the transparency of the source color.

  The specific blending factors used can be configured based on the desired effect.

- $\odot$ represents the operator used to combine the source and destination colors. Common operators include addition, subtraction, minimum, and maximum.

The well knwon α-blending is achieved by using the following blending factors and operator:

- $F_{\text{source}} =$ `GL_SRC_ALPHA`
- $F_{\text{destination}} =$ `GL_ONE_MINUS_SRC_ALPHA`
- $\odot$ is the addition operator.

This is a common blending mode used to achieve transparency effects. However, there are many other blending modes that can be used to achieve different effects. For instance, the following blending mode can be used to count how many fragments are rendered at each pixel, which can be useful for various effects such as outlining objects or creating shadows:

- $F_{\text{source}} =$ `GL_ONE`
- $F_{\text{destination}} =$ `GL_ONE`
- $\odot$ is the addition operator.

3.2. Programmable Pipeline

The programmable pipeline is a modern graphics pipeline that allows developers to write custom shaders to control the behavior of each stage of the pipeline. This provides greater flexibility and control over the rendering process, allowing for more complex and realistic graphics. The programmable pipeline consists of several stages, including vertex shading, tessellation, geometry shading, fragment shading, and more. Each stage can be programmed using a shading language such as GLSL (OpenGL Shading Language) or HLSL (High-Level Shading Language). The programmable pipeline has largely replaced the fixed-function pipeline in modern graphics hardware, as it allows for more advanced rendering techniques and greater performance.

4. Color

During this chapter, we will explore the concept of color. In order to do so, we firstly need to understand what "light" is. Light is a form of electromagnetic radiation, which are two perpendicular transverse waves that propagate through space. These waves consist of two components:

- The electric field wave, which oscillates in a plane perpendicular to the direction of propagation.
- The magnetic field wave, which also oscillates in a plane perpendicular to the direction of propagation, but is perpendicular to the electric field wave.

The last degree of freedom is the polarization of the light, which describes the orientation of the electric field wave. A really important property of light is its wavelength, which divides the electromagnetic spectrum into different regions, from radio waves (long wavelength, less energy) to gamma rays (short wavelength, more energy). The visible spectrum, which is the range of wavelengths that human eyes can perceive, ranges from approximately $380nm$ (violet) to $750nm$ (red). The light we perceive is actually a superposition of different wavelengths of light, called a *spectral energy distribution* (represented as a function of wavelength vs. intensity).

Once we have understood what light is, we can define color as the perception of light by the human eye (which will be explained in detail in the next section). The color we percieve depends on the type of object we are looking at:

- If the object is a light source, the color we perceive is determined by the spectral energy distribution of the light emitted by the source.
- If the object is a non-light source, the color we perceive is determined by the spectral energy distribution of the light reflected by the object. Of all the incoming wavelengths of light that hit the object, some are absorbed (and maybe converted to heat) and some are reflected. The reflected wavelengths are the ones that determine the color we perceive.

4.1. Technical Color Models

For biological purposes (which we will discuss in the following section), colors are defined by a superposition of three primary colors, which are red, green, and blue (as if they were the usual base of $\mathbb{R}^3$). These three colors (and no other) were chosen because of the three type of cone cells in the human eye (explained in the following section). The similarity between these three primary colors and the three dimensions

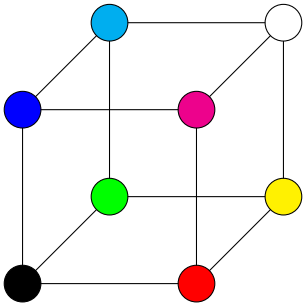


Figura 4.1: The RGB color cube.

of a vector space is represented in the RGB color cube in Figure 4.1. Each vertex of the cube represents a color, where the coordinates of the vertex correspond to the intensity of red, green, and blue components.

4.1.1. Additive Color Model: RGB

This color model is called *additive* because the origin is the absence of color (black), and the colors are created by combining the primary colors (red, green, blue) in different intensities. When all three primary colors are combined at full intensity, we get white.

This model is commonly used in sources of light, such as computer monitors, televisions, and smartphones, where colors are created by emitting light. In all these cases, the origin is black (the absence of light), and the colors are created by adding light from the primary colors.

4.1.2. Subtractive Color Model: CMY

This color model is called *subtractive* because the origin is the presence of all colors (white), and the colors are created by mixing the secondary colors (cyan, magenta, yellow) in different intensities. This is done because each of the secondary colors absorbs (or subtracts) one of the primary colors (the one that is not present in the secondary color; the one opposite to it in the RGB color cube):

- Cyan (which is in fact a mixture of green and blue) absorbs red.
- Magenta (which is in fact a mixture of red and blue) absorbs green.
- Yellow (which is in fact a mixture of red and green) absorbs blue.

Therefore, when all three secondary colors are combined at full intensity, we get black (the absence of light). This model is commonly used in printing, where the source is a non-light source (the paper) that reflects all the incoming light (white), and the colors are created by subtracting light. The conversion between the RGB and CMY color models is straightforward, as they are complementary to each other:

$$(R, G, B) = 1 - (C, M, Y)$$
$$(C, M, Y) = 1 - (R, G, B)$$

CMYK Color Model

The CMYK color model is an extension of the CMY color model, where the K component stands for "key" (black)[1]. The reason for adding the K component is that the pigments in real life are not perfect, and when we mix the three secondary colors (cyan, magenta, yellow) at full intensity, we get a dark brown color instead of black. In addition, creating a black ink is much cheaper than creating a black color by mixing the three secondary colors, so it is more efficient to use a separate black ink.

As black absorves all the incoming light, the conversion between the RGB and CMYK color models is as follows:

$$\begin{aligned} K &= \text{mín}\{1-R, 1-G, 1-B\} \\ (C, M, Y) &= 1 - (R, G, B) - K \end{aligned}$$

That way, we reduce the amount of expensive (C, M, Y) ink used, even not using one of them at all (the one that corresponds to the minimum of $1-R$, $1-G$, and $1-B$).

4.2. Perceptual Color Models

Some color models are designed to be more intuitive for humans, and to better represent how we perceive color. These models are called *perceptual* color models, and they are based on how do we perceive color. it is important to first understand how the human eye perceives light and color. Light first enters the eye through the *cornea*, a first *lens* that protects the eye and focus light. The light then passes through the *pupil*, which is a hole that can adjust its size to control the amount of light that enters the eye. The *iris*, which is the colored part of the eye, controls the size of the pupil. After the light passes through the pupil, it is slightly focused by the *lens* onto the *retina*, which is a layer of light-sensitive cells (photoreceptors) at the back of the eye. The information captured by the photoreceptors is then transmitted to the brain via the optic nerve, where it is processed and interpreted as visual information. There are two main types of photoreceptors in the human eye: rods and cones.

Rods are responsible for vision in low-light conditions. They are highly sensitive to light but do not provide color information. Rods are more numerous ($\sim 120 \cdot 10^6$) than cones and are primarily located in the peripheral regions of the retina.

Cones are responsible for vision in bright-light conditions and for color perception. They are less sensitive to light than rods but can detect a wide range of colors. Cones are concentrated in the central part of the retina, particularly in the fovea, which is responsible for sharp central vision. There are much less cones ($\sim 6 \cdot 10^6$) than rods in the human eye. There are three types of cones, each sensitive to different wavelengths of light:

[1]The letter "B" is already used for blue in the RGB color model, so we use "K" to avoid confusion.

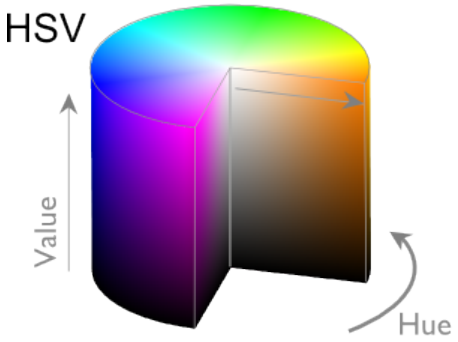


Figura 4.2: The HSV color model.

- S-cones (short-wavelength cones). Only 2 % of the cones are S-cones, which are most sensitive to blue light.
- M-cones (medium-wavelength cones). About 32 % of the cones are M-cones, which are most sensitive to green light.
- L-cones (long-wavelength cones). The majority of cones, about 64 %, are L-cones, which are most sensitive to red light.

This leads to the concept of *metamerism*, which is the phenomenon where different spectral energy distributions can produce the same color perception. This is because the human eye only has three types of cones, and therefore can only perceive colors as a combination of three primary colors (red, green, blue). As a result, different combinations of wavelengths can stimulate the cones in the same way, leading to the same color perception.

- For instance, a pure yellow light (which has a wavelength of around $580nm$) can stimulate the L-cones and M-cones in a way that is similar to a combination of red and green light (which also stimulates the L-cones and M-cones). Therefore, both the pure yellow light and the combination of red and green light can produce the same color perception of yellow, which our brain cannot distinguish between them.

This leads to the concept of *trichromacy*, which is the idea that any color can be represented as a combination of three primary colors. This is the basis for the RGB color model. However, there are other perceptual color models that are designed to be more intuitive for humans, and to better represent how we perceive color.

4.2.1. HSV Color Model

The HSV color model (Hue, Saturation, Value) is a cylindrical representation of the RGB color model, represented in Figure 4.2. It is designed to be more intuitive for humans, as it separates the color information (hue) from the intensity information (saturation and value).

- Hue ($[0 - 360[$): represents the dominant wavelength of the color, which corresponds to the type of color (red, green, blue, etc.). It is represented as an angle around the central axis of the cylinder, where 0° corresponds to red, 120° corresponds to green, and 240° corresponds to blue.

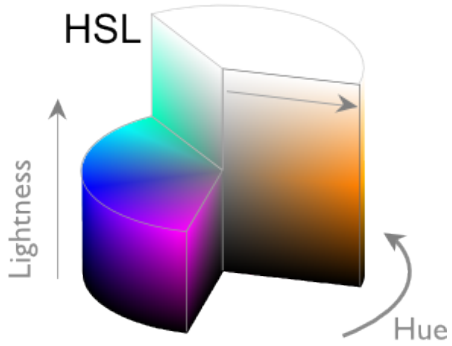


Figura 4.3: The HSL color model.

- Saturation ($[0-1]$): represents the purity of the color, which corresponds to how much the color is diluted with white light. A saturation of 0 corresponds to a completely desaturated color (white), while a saturation of 1 corresponds to a fully saturated color (the pure hue).

- Value ($[0-1]$): represents the brightness of the color, which corresponds to how much light is emitted (or reflected) by the color. A value of 0 corresponds to a completely dark color (black), while a value of 1 corresponds to a fully bright color (the saturated hue).

4.2.2. HSL Color Model

The HSL color model (Hue, Saturation, Lightness) is another cylindrical representation of the RGB color model, represented in Figure 4.3. It is similar to the HSV color model, but:

- In HSV, the cylinder goes from completely dark (black) at the bottom to the pure hue at the top.

- In HSL, the cylinder goes from completely dark (black) at the bottom to completely bright (white) at the top.

The hue and saturation components are the same as in the HSV color model, but the lightness component is different from the value component in the HSV color model.

- Lightness ($[0-1]$): represents the brightness of the color, which corresponds to how much light is emitted (or reflected) by the color. A lightness of 0 corresponds to a completely dark color (black), while a lightness of 1 corresponds to a completely bright color (white).

4.3. Other Color Models

Apart from the additive and subtractive color models, and the perceptual color models, there are other color models that are designed for specific purposes.

4.3.1. YUV Color Model

As it has already been explained, the human eye has much more rods than cones, and therefore is much more sensitive to changes in brightness than to changes in color. This idea is the basis for the YUV color model, which separates the luminance information from the chrominance information. It has three components:

- Y (luminance): represents the brightness of the color, which corresponds to how much light is emitted (or reflected) by the color.
- U (u-chrominance): represents the color information, which corresponds to the difference between the *blue* component and the luminance.
- V (v-chrominance): represents the color information, which corresponds to the difference between the *red* component and the luminance.

Given that the YUV color model separates the luminance information from the chrominance information, it is the most efficient color model for compressing images and videos, as it allows us to reduce the amount of chrominance information (which is less important for human perception) without affecting the luminance information (which is more important for human perception). This is why it is widely used in video compression formats such as MPEG and JPEG.

4.3.2. XYZ Color Model

In 1931, an important experiment was conducted by the International Commission on Illumination (CIE) to create a standard color space that could be used to represent all perceivable colors. The experiment is described in Figure 4.4, where we can see that the participants were seeing a screen that was divided into two halves.

- On the left half of the screen, there was a test color (which could be any color in the visible spectrum). This was generated by a monochromatic light source that could produce light of any wavelength in the visible spectrum.
- On the right half of the screen, there were three primary colors (red, green, blue) that could be adjusted in intensity by the participants.

The participants were asked to adjust the intensities of the three primary colors on the right half of the screen until they matched the test color on the left half of the screen. By doing this for a large number of test colors, the CIE wanted to specify how much of each primary color was needed to match any given color in the visible spectrum. However, a problem appeared.

- For some test colors, it was not possible to match them by adding the three primary colors (red, green, blue) on the right half of the screen.

  The only way to match those colors was by adding a second light source on the left half of the screen (where the test color was) that only emitted red light. Adding red light on the left half of the screen was equivalent to subtracting red light on the right half of the screen, so some colors actually needed a negative amount of red light on the right half of the screen to be matched.

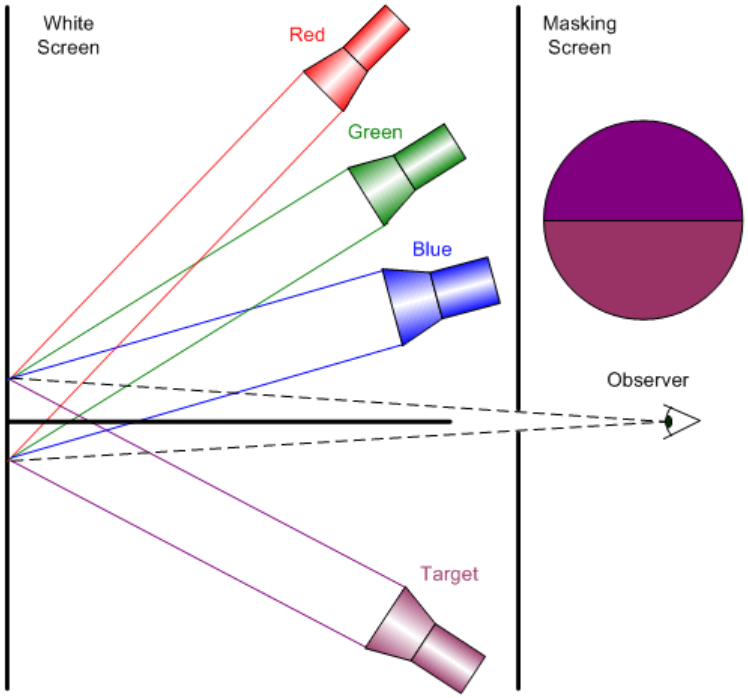


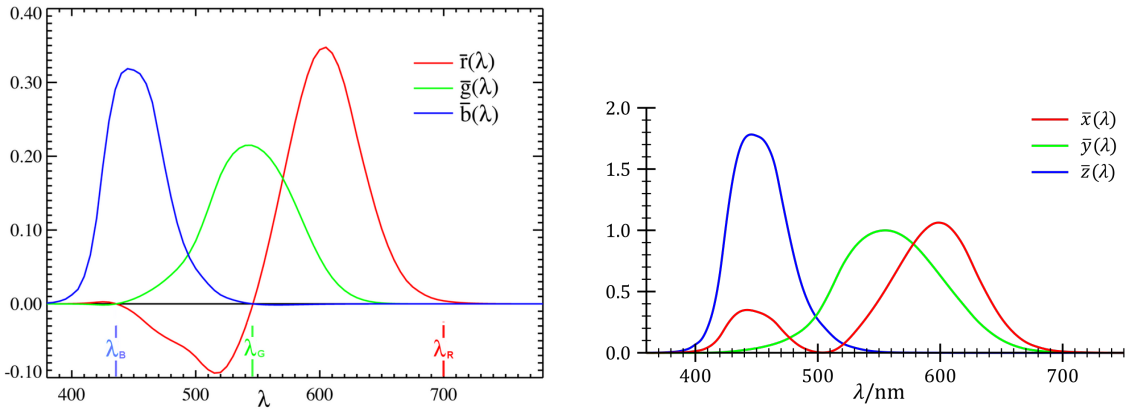
Figura 4.4: The CIE 1931 color matching experiment.

After this experiment, the color matching functions were defined, which specify how much of each primary color is needed to match any given color in the visible spectrum. These human-made color matching functions are represented in Figure 4.5a, where the problem of negative values can be observed (the red color matching function goes below zero for some wavelengths).

After defining these three functions, $\overline{r}(\lambda)$, $\overline{g}(\lambda)$, and $\overline{b}(\lambda)$, the CIE proposed the RGB components of a color as the integrals of the the spectral energy distribution (denoted as $S(\lambda)$) weighed by the color matching functions:

$$R = \int_{\lambda} S(\lambda)\overline{r}(\lambda)\, d\lambda$$
$$G = \int_{\lambda} S(\lambda)\overline{g}(\lambda)\, d\lambda$$
$$B = \int_{\lambda} S(\lambda)\overline{b}(\lambda)\, d\lambda$$

However, this had a big problem: some of the colors would have negative RGB components, which is not possible to represent in a color model. To solve this problem, the CIE defined a new set of color matching functions, called the XYZ color matching functions, which are linear combinations of the original RGB color matching functions. The XYZ color matching functions are represented in Figure 4.5b, where we can see that all the values are positive. The XYZ color model is defined



(a) RGB Color matching functions.

(b) XYZ Color matching functions.

Figura 4.5: Color matching functions.

by the following equations:

$$X = \int_{\lambda} S(\lambda)\overline{x}(\lambda)\, d\lambda$$
$$Y = \int_{\lambda} S(\lambda)\overline{y}(\lambda)\, d\lambda$$
$$Z = \int_{\lambda} S(\lambda)\overline{z}(\lambda)\, d\lambda$$

It should be noted that converting from the XYZ color model to the RGB color model is straightforward, as it simply involves a linear transformation. Finding that matrix was however not trivial. Lastly, the components XYZ should be normalized to be in the range $[0, 1]$ by dividing them by the sum of the components:

$$x = \frac{X}{X + Y + Z}$$
$$y = \frac{Y}{X + Y + Z}$$
$$z = \frac{Z}{X + Y + Z}$$

These normalized components are called the chromaticity coordinates. When represented in a 3D cube, a cross-section by the plane $z = 0$ gives us the CIE 1931 chromaticity diagram, shown Figure 4.6, where we can see that all the perceivable colors are represented as points in the diagram, and the spectral colors (the colors that correspond to a single wavelength) are represented as a curve around the edge of the diagram, where the wavelength is indicated in nanometers.

It should be noted that the XYZ color model is not designed to be intuitive for humans, but its main advantage is that it is able to represent all the perceivable colors, as it is based on the human color matching functions. Therefore, it is used as a reference color model for defining other color models, such as the RGB color model, which can be defined as a linear transformation of the XYZ color model. In addition, the XYZ color model is also used in color management systems to convert between different color spaces and to ensure color consistency across different devices.

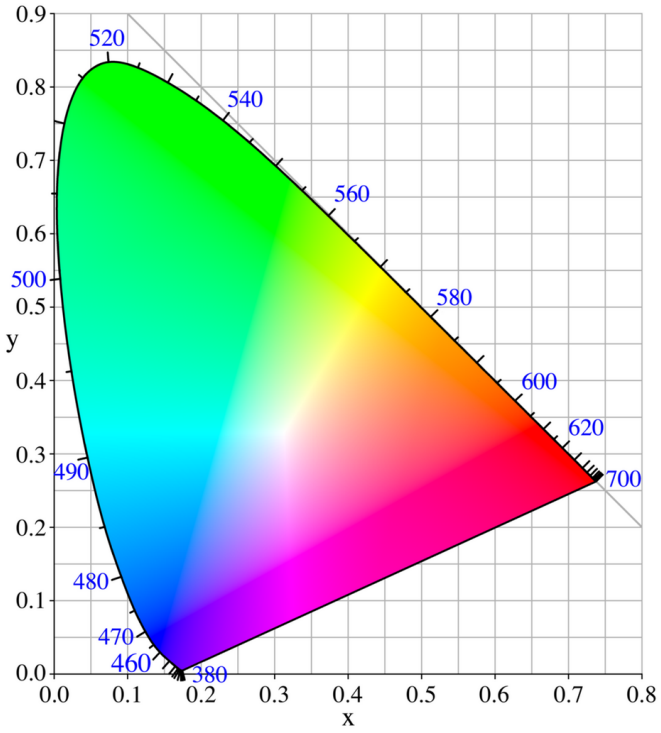


Figura 4.6: The CIE 1931 chromaticity diagram.

5. Splines

In computer graphics, we often need to create smooth curves that can be easily manipulated. Curves have 3 different representations:

- Explicit: $y = f(x)$

  As a disadvantage, explicit curves cannot represent vertical lines, so they are not suitable for all types of curves.

- Implicit: $f(x, y) = 0$

  As a disadvantage, implicit curves can be more difficult to work with, especially when it comes to rendering and manipulating them.

- Parametric: $x = f(t), y = g(t)$

  Parametric curves are often preferred in computer graphics because they allow for more flexibility and control over the shape of the curve. They can represent a wider variety of curves, including those that cannot be represented in explicit form, such as vertical lines. Additionally, the problem is simplified, as we only consider one dimension at a time.

The main goal when working with curves is to interpolate some given points, which means finding a curve that passes through those points. Parametric curves are often used with the goal of interpolation, as they allow us to simply the problem by considering each coordinate separately, which is what we will do in the following sections. Some desired properties of the interpolating curve are:

- C^0: The curve is continuous, which means that there are no gaps or jumps in the curve.

- C^1: A curve is parametric continuos C^1 if the curve is derivative continuous, which means that the curve has a well-defined tangent at every point, and there are no sharp corners or cusps in the curve.

  In each node, the curve is C^1 if the tangent of the curve is the same when approaching the node from the left and from the right.

- G^1: A curve is geometric continuous G^1 if, at every node, the tangent of the curve has the same direction when approaching the node from the left and from the right, but it can have different magnitudes.

- C^2: The curve is second derivative continuous, which means that the curve has a well-defined curvature at every point, and there are no inflection points.

Another important property of the curve is the *Convex Hull Property*, which states that the curve is contained within the convex hull of its control points. Its control points are not only the points that we want to interpolate, but also the points that we use to define the shape of the curve. The convex hull of a set of points is the smallest convex shape that contains all of the points. The convex hull property is desirable because, when it is satisfied, in order to calculate intersections between the curve and other objects, we can first calculate the intersection between the convex hull and the other object, which is much easier to compute. If there is no intersection between the convex hull and the other object, then there is no intersection between the curve and the other object, so we can avoid performing more complex calculations. The needed and sufficient condition for a curve to satisfy the convex hull property is:

- The curve is a linear combination of its control points, which means that the curve can be expressed as a weighted sum of its control points, where the weights are non-negative and sum up to 1.

  $$P(t) = \sum_{i=0}^{n} w_i(t) P_i$$

  where:

  - $\sum_{i=0}^{n} w_i(t) = 1$ for all t.
  - $w_i(t) \geqslant 0$ for all i and for all t.

5.1. Polinomial Interpolation

Given a set of n points, the aim is to find a polynomial of degree $n-1$ (which is the lowest degree possible) that passes through all of those points. The polynomial can be expressed as:

$$P(t) = \sum_{i=0}^{n-1} a_i t^i$$

where a_i are the coefficients of the polynomial and t^i are the monomials. There are different methods to find the coefficients a_i.

5.1.1. Undetermined Coefficients

This method involves setting up a system of equations based on the given points and solving for the coefficients a_i. For each point (t_j, y_j), we can substitute it into the polynomial equation to get:

$$y_j = \sum_{i=0}^{n-1} a_i t_j^i$$

This results in a system of n equations with n unknowns (the coefficients a_i). We can solve this system using methods such as Gaussian elimination or matrix inversion.

5.1.2. Lagrange Interpolation

The Lagrange interpolation method constructs the polynomial using a linear combination of basis polynomials. The Lagrange basis polynomial for the i-th point is defined as:

$$L_i(t) = \prod_{j=0, j\neq i}^{n-1} \frac{t - t_j}{t_i - t_j}$$

These basis polynomials have the property that:

$$L_i(t_j) = \delta_{ij} = \begin{cases} 1 & \text{if } i = j \\ 0 & \text{if } i \neq j \end{cases}$$

Therefore, the interpolating polynomial can be expressed as:

$$P(t) = \sum_{i=0}^{n-1} y_i L_i(t)$$

This method is straightforward and does not require solving a system of equations, but it can be computationally expensive for large n due to the need to compute the basis polynomials.

5.1.3. Runge's Phenomenon

When using high-degree polynomials for interpolation, especially with equally spaced points, we can encounter Runge's phenomenon, which is characterized by large oscillations at the edges of the interval. In the Figure 5.1 we can see an example of this phenomenon, where the interpolating polynomial oscillates significantly, even though it passes through all the given points (being therefore a perfect fit). This is a major drawback of using high-degree polynomials for interpolation, and it motivates the use of alternative methods such as splines, which we will discuss in the next section.

5.2. Splines

To overcome the issues associated with high-degree polynomial interpolation, we can use splines. A spline is a piecewise polynomial function that is defined on a set of intervals, with each interval having its own polynomial. Therefore, when there is a lot of points to interpolate, a common practice is to divide the interval into different segments (when each segment contains two points) and use a low-degree polynomial to interpolate the points in each segment. After that, we can connect the segments together to form a smooth curve. This approach allows us to achieve a smooth curve without the oscillations associated with high-degree polynomials. During the following sections, we will discuss different types of splines and their properties.

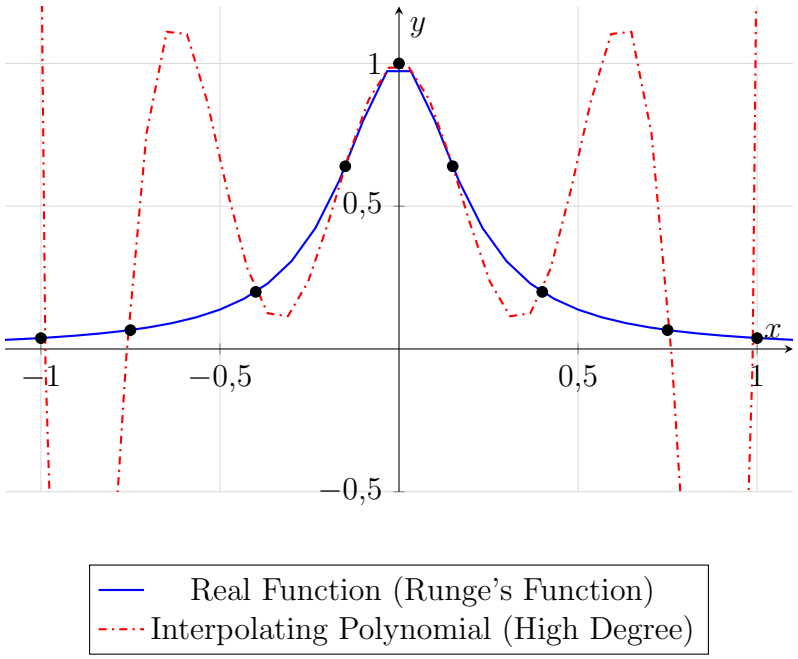


Figura 5.1: Runge's Phenomenon

5.2.1. Constant Splines

The simplest type of spline is the constant spline, which is a piecewise constant function. In each segment, the function takes the value of the starting point of the segment. Mathematically, it can be defined as:

$$S(t) = y_i \quad \text{for } t_i \leqslant t < t_{i+1}$$

An example of a constant spline is shown in the Figure 5.2. As we can see, the curve is not even continuous, as it has jumps at the knots, so more complex splines are usually preferred.

5.2.2. Linear Splines

A linear spline is a piecewise linear function that connects the given points with straight lines. In each segment, the function is defined as the linear interpolation between the two points that define the segment. Mathematically, it is defined as:

$$S(t) = y_i + \frac{y_{i+1} - y_i}{t_{i+1} - t_i}(t - t_i) \quad \text{for } t_i \leqslant t < t_{i+1}$$

An example of a linear spline is shown in the Figure 5.3. As we can see, the curve is continuous, but it is not smooth, as it has sharp corners at the knots. This is why we usually prefer to use cubic splines, which we will discuss in the next section.

During the following sections, we will discuss different types of cubic splines. They are the most commonly used type of splines, as they provide a good balance between smoothness and computational efficiency. Without loss of generality, we will consider the interval $t \in [0, 1]$. In each segment, the function is defined as a cubic polynomial that interpolates the two points that define the segment. However,

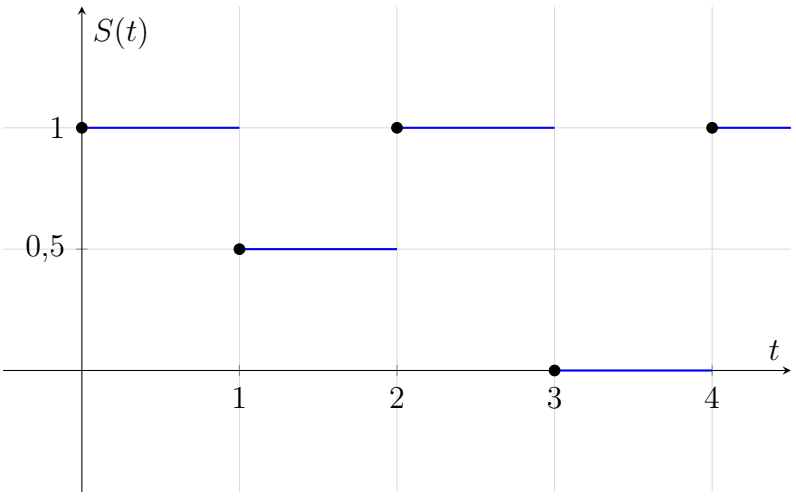


Figura 5.2: Constant Spline Example

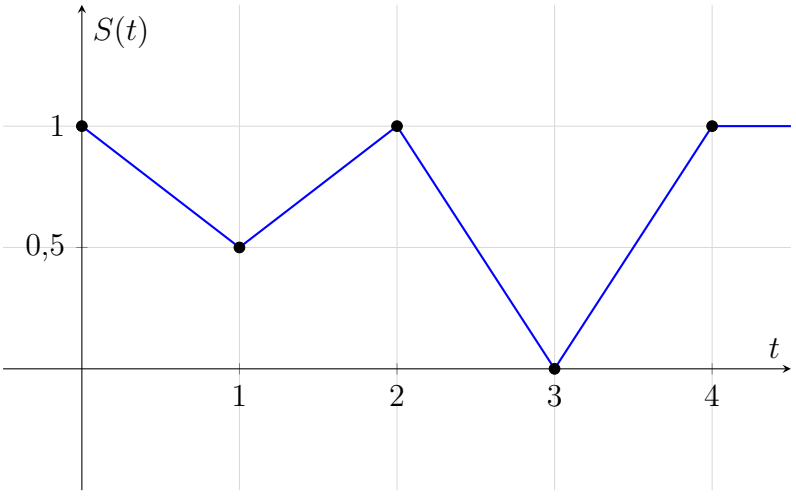


Figura 5.3: Linear Spline Example

a third degree polynomial has 4 coefficients, so we need to impose 2 additional conditions to be able to solve for those coefficients. Different types of cubic splines are defined based on the additional conditions that we impose.

5.2.3. Hermite Cubic Splines

Observación. This interactive applet shows how the Hermite splines work, and will help the user to understand this concept.

A Hermite cubic spline is a piecewise cubic function that is defined by the values of the function and its derivative at the endpoints of each segment. In other words, if the interpolating polynomial is P, we have the following conditions:

$$\begin{aligned} P(0) &= y_0 \\ P(1) &= y_1 \\ P'(0) &= m_0 \\ P'(1) &= m_1 \end{aligned}$$

where y_0 and y_1 are the values of the function at the endpoints of the segment, and m_0 and m_1 are the values of the derivative[1] at the endpoints of the segment. After solving and rearranging the equations, we can express the Hermite cubic spline as:

$$P(t) = H_{y_0}(t)y_0 + H_{y_1}(t)y_1 + H_{m_0}(t)m_0 + H_{m_1}(t)m_1$$

where $H_{y_0}(t)$, $H_{y_1}(t)$, $H_{m_0}(t)$ and $H_{m_1}(t)$ are the Hermite basis functions, shown in Figure 5.4 and defined as:

$$\begin{aligned} H_{y_0}(t) &= (1-t)^2(1+2t) \\ H_{y_1}(t) &= (3-2t)t^2 \\ H_{m_0}(t) &= t(1-t)^2 \\ H_{m_1}(t) &= t^2(t-1) \end{aligned}$$

Demostración. A general cubic polynomial can be expressed as:

$$P(t) = \sum_{i=0}^{3} a_i t^i = \begin{pmatrix} 1 & t & t^2 & t^3 \end{pmatrix} \begin{pmatrix} a_0 \\ a_1 \\ a_2 \\ a_3 \end{pmatrix}$$

Our aim is to find the coefficients a_i such that the conditions of the Hermite cubic spline are satisfied. We can express the conditions in matrix form as:

$$\begin{pmatrix} 1 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 \\ 0 & 1 & 0 & 0 \\ 0 & 1 & 2 & 3 \end{pmatrix} \begin{pmatrix} a_0 \\ a_1 \\ a_2 \\ a_3 \end{pmatrix} = \begin{pmatrix} y_0 \\ y_1 \\ m_0 \\ m_1 \end{pmatrix}$$

[1] The corresponding coordinate of the derivative, as we are working with parametric equations.

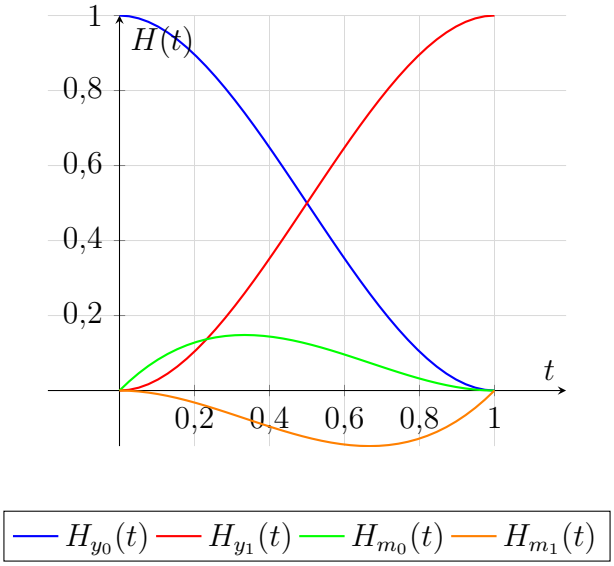


Figura 5.4: Hermite Basis Functions

We can solve this system of equations to find the coefficients a_i. Inverting the matrix on the left-hand side, we get:

$$\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ -3 & 3 & -2 & -1 \\ 2 & -2 & 1 & 1 \end{pmatrix} \begin{pmatrix} y_0 \\ y_1 \\ m_0 \\ m_1 \end{pmatrix} = \begin{pmatrix} a_0 \\ a_1 \\ a_2 \\ a_3 \end{pmatrix}$$

Therefore, the polynomial can be expressed as:

$$P(t) = \begin{pmatrix} 1 & t & t^2 & t^3 \end{pmatrix} \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ -3 & 3 & -2 & -1 \\ 2 & -2 & 1 & 1 \end{pmatrix} \begin{pmatrix} y_0 \\ y_1 \\ m_0 \\ m_1 \end{pmatrix}$$

Multiplying the first two matrices, we get:

$$P(t) = \begin{pmatrix} H_{y_0}(t) & H_{y_1}(t) & H_{m_0}(t) & H_{m_1}(t) \end{pmatrix} \begin{pmatrix} y_0 \\ y_1 \\ m_0 \\ m_1 \end{pmatrix}$$

where $H_{y_0}(t)$, $H_{y_1}(t)$, $H_{m_0}(t)$ and $H_{m_1}(t)$ are the Hermite basis functions presented above. □

Once each segment is defined, we can connect them together imposing C^1 continuity at the knots, which means that the derivative of the curve is continuous at the knots. This type of spline has however some disadvantages:

- It does not satisfy the convex hull property, as the basis functions can take negative values.

- It is not only controlled by points but also by the derivatives at the endpoints of each segment (vectors). This has several disadvantages:
  - It is not intuitive to control the shape of the curve by manipulating the derivatives at the endpoints of each segment.
  - It requires special care when transforming the curve. Points can always be transformed using the same transformation, but vectors need to be transformed using the inverse transpose of the transformation, which can be more complex to implement.

Catmull-Rom Splines

Hermite cubic splines require us to specify the derivative at the endpoints of each segment, which can be difficult to control. A common practice is to use the Catmull-Rom spline, which is a type of Hermite cubic spline where the derivative at each node is defined as:

$$\begin{aligned} m_0 &= y_1 - y_0 \\ m_i &= \frac{(y_{i+1} - y_i) + (y_i - y_{i-1})}{2} = \frac{y_{i+1} - y_{i-1}}{2} \quad \text{for } i = 1, \dots, n-1 \\ m_n &= y_n - y_{n-1} \end{aligned}$$

It should be noted that it simply defines the derivative at each node as the average of the slopes of the segments that connect the node to its neighbors, which is a simple and intuitive way to control the shape of the curve.

5.2.4. Bezier Cubic Splines

Observación. This interactive applet shows how the Bezier splines work, and will help the user to understand this concept.

A Bezier cubic spline is a piecewise cubic function that is defined by four control points: the two endpoints of the segment and two additional control points that determine the shape of the curve. The conditions for a Bezier cubic spline are:

$$\begin{aligned} P(0) &= y_0 \\ P(1) &= y_3 \\ P'(0) &= 3(y_1 - y_0) \\ P'(1) &= 3(y_3 - y_2) \end{aligned}$$

where y_0 and y_3 are the endpoints of the segment, and y_1 and y_2 are the control points that determine the shape of the curve. The two additional conditions simply represent the derivative[2] of the curve at the endpoints (scaled by 3[3]), which is determined by the control points. After solving and rearranging the equations, we can express the Bezier cubic spline as:

$$P(t) = B_{y_0}(t)y_0 + B_{y_1}(t)y_1 + B_{y_2}(t)y_2 + B_{y_3}(t)y_3$$

[2]It should be noted that, in fact, it is not the derivate but its equivalent in only one dimension.

[3]The factor of 3 is used to have a constant speed along the curve. Others values could be used achieve other known effects as *ease-in* or *ease-out*.

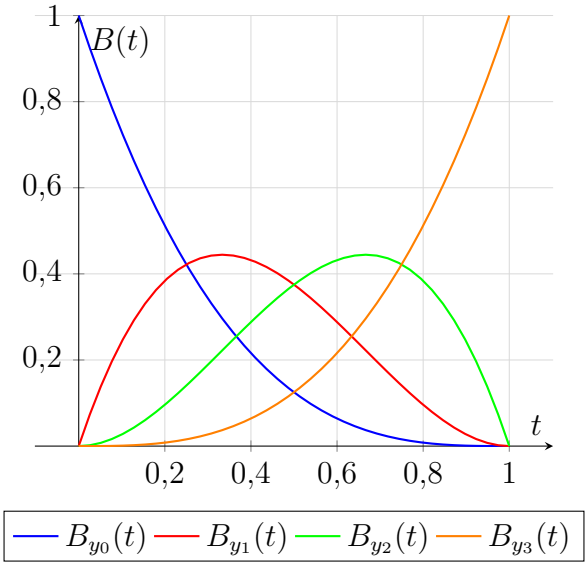


Figura 5.5: Bezier Basis Functions

where $B_{y_0}(t)$, $B_{y_1}(t)$, $B_{y_2}(t)$ and $B_{y_3}(t)$ are the Bezier basis functions, shown in Figure 5.5 and defined as:

$$\begin{aligned} B_{y_0}(t) &= (1-t)^3 \\ B_{y_1}(t) &= 3t(1-t)^2 \\ B_{y_2}(t) &= 3t^2(1-t) \\ B_{y_3}(t) &= t^3 \end{aligned}$$

In general, the Bezier basic functions can be defined for any degree n as:

$$B_{y_i}(t) = \binom{n}{i} t^i (1-t)^{n-i}$$

Demostración. A general cubic polynomial can be expressed as:

$$P(t) = \sum_{i=0}^{3} a_i t^i = \begin{pmatrix} 1 & t & t^2 & t^3 \end{pmatrix} \begin{pmatrix} a_0 \\ a_1 \\ a_2 \\ a_3 \end{pmatrix}$$

Our aim is to find the coefficients a_i such that the conditions of the Bezier cubic spline are satisfied.

$$\begin{aligned} P(0) &= a_0 = y_0 \\ P(1) &= a_0 + a_1 + a_2 + a_3 = y_3 \\ P'(0) &= a_1 = 3y_1 - 3y_0 \\ P'(1) &= a_1 + 2a_2 + 3a_3 = 3y_3 - 3y_2 \end{aligned}$$

Rearranging the equations and expressing them in matrix form, we get:

$$\begin{pmatrix} 1 & 0 & 0 & 0 \\ 1 & 1/3 & 0 & 0 \\ 1 & 2/3 & 1/3 & 0 \\ 1 & 1 & 1 & 1 \end{pmatrix} \begin{pmatrix} a_0 \\ a_1 \\ a_2 \\ a_3 \end{pmatrix} = \begin{pmatrix} y_0 \\ y_1 \\ y_2 \\ y_3 \end{pmatrix}$$

Inverting the matrix on the left-hand side, we get:

$$\begin{pmatrix} 1 & 0 & 0 & 0 \\ -3 & 3 & 0 & 0 \\ 3 & -6 & 3 & 0 \\ -1 & 3 & -3 & 1 \end{pmatrix} \begin{pmatrix} y_0 \\ y_1 \\ y_2 \\ y_3 \end{pmatrix} = \begin{pmatrix} a_0 \\ a_1 \\ a_2 \\ a_3 \end{pmatrix}$$

Therefore, the polynomial can be expressed as:

$$P(t) = \begin{pmatrix} 1 & t & t^2 & t^3 \end{pmatrix} \begin{pmatrix} 1 & 0 & 0 & 0 \\ -3 & 3 & 0 & 0 \\ 3 & -6 & 3 & 0 \\ -1 & 3 & -3 & 1 \end{pmatrix} \begin{pmatrix} y_0 \\ y_1 \\ y_2 \\ y_3 \end{pmatrix}$$

Multiplying the first two matrices, we get:

$$P(t) = \begin{pmatrix} B_{y_0}(t) & B_{y_1}(t) & B_{y_2}(t) & B_{y_3}(t) \end{pmatrix} \begin{pmatrix} y_0 \\ y_1 \\ y_2 \\ y_3 \end{pmatrix}$$

where $B_{y_0}(t)$, $B_{y_1}(t)$, $B_{y_2}(t)$ and $B_{y_3}(t)$ are the Bezier basis functions presented above. □

Once each segment is defined, we can connect them together imposing C^0 continuity at the knots. Some specific characteristics of Bezier cubic splines are:

- They do satisfy the convex hull property (as the basis functions are non-negative and sum up to 1).
- They are only controlled by points, which is more intuitive than controlling the shape of the curve by manipulating the derivatives at the endpoints of each segment, as in Hermite cubic splines.
- They do not require special care when transforming the curve, as all the control points can be transformed using the same transformation, unlike in Hermite cubic splines where we need to transform points and vectors differently.
- For each knot, if its previous and next control points are collinear with the knot, then the curve is G^1 continuous at that knot, as the tangent of the curve has the same direction when approaching the knot from the left and from the right.

De Casteljau's Algorithm

De Casteljau's algorithm is a recursive method for evaluating Bezier curves at a given parameter value t. It is based on the principle of linear interpolation and can be used to compute points on the Bezier curve efficiently, and can also be used to graphically represent the curve. Let's see an example of how De Casteljau's algorithm works for a cubic Bezier curve defined by control points y_0, y_1, y_2 and y_3.

$$\begin{aligned} P(t) &= B_{y_0}(t)y_0 + B_{y_1}(t)y_1 + B_{y_2}(t)y_2 + B_{y_3}(t)y_3 \\ &= (1-t)^3 y_0 + 3t(1-t)^2 y_1 + 3t^2(1-t)y_2 + t^3 y_3 \\ &= (1-t)\left\{(1-t)\left[(1-t)y_0 + ty_1\right] + t\left[(1-t)y_1 + ty_2\right]\right\} + \\ &\quad + t\left\{(1-t)\left[(1-t)y_1 + ty_2\right] + t\left[(1-t)y_2 + ty_3\right]\right\} \end{aligned}$$

As we can see, several interpolations are made:

- First, we interpolate between the control points to get three new points:

$$\begin{aligned} y_{01} &= (1-t)y_0 + ty_1 \\ y_{12} &= (1-t)y_1 + ty_2 \\ y_{23} &= (1-t)y_2 + ty_3 \end{aligned}$$

  where y_{ij} represents the point obtained by interpolating between the control points y_i and y_j. It should be noted that y_{12} is obtained twice.

- Then, we interpolate between the new points to get two new points:

$$\begin{aligned} y_{(01)(12)} &= (1-t)y_{01} + ty_{12} \\ y_{(12)(23)} &= (1-t)y_{12} + ty_{23} \end{aligned}$$

- Finally, we interpolate between the last two points to get the point on the curve corresponding to the parameter value t:

$$P(t) = (1-t)y_{(01)(12)} + ty_{(12)(23)}$$

Doing this process several times for different values of t allows us to compute multiple points on the curve, which can be used to graphically represent the curve.

5.2.5. B-Spline Cubic Splines

Hermite and Bezier cubic splines have some disadvantages, such as not being able to achieve C^2 continuity at the knots. B-spline cubic splines address this issue but resign an important property: they do not interpolate the control points, but only approximate them. A B-spline cubic spline is a piecewise cubic function that is defined by a set of control points. In order to achieve C^2 continuity, the perspective is not to define the curve by independent segments, but by segments *whose control points overlap*. The polynomial that defines the i-th segment is:

$$P_i(t) = \sum_{k=0}^{3} y_{i+k} B_k(t)$$

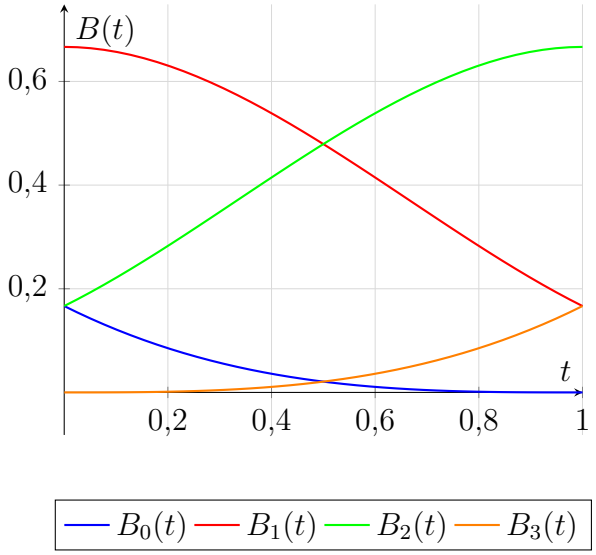


Figura 5.6: B-spline Basis Functions

where y_{i+k} are the control points and $B_k(t)$ are the B-spline basis functions, shown in Figure 5.6 and defined as:

$$
\begin{aligned}
B_0(t) &= 1/6 \cdot (1-t)^3 \\
B_1(t) &= 1/6 \cdot (3t^3 - 6t^2 + 4) \\
B_2(t) &= 1/6 \cdot (-3t^3 + 3t^2 + 3t + 1) \\
B_3(t) &= 1/6 \cdot t^3
\end{aligned}
$$

By the definition, it is perfectly clear that the control points of each segment overlap with the control points of the previous and next segments, as:

- During the i-th segment, the control points are y_i, y_{i+1}, y_{i+2} and y_{i+3}.
- During the $(i+1)$-th segment, the control points are y_{i+1}, y_{i+2}, y_{i+3} and y_{i+4}. Three of the control points are shared with the previous segment, which allows us to achieve C^2 continuity.
- It should also be taken into account that n control points define $n-3$ segments, as the i-th segment uses up to the $i+3$-th control point.

Let's adress know how to get to the basis functions. The conditions imposed to the B-spline cubic spline are:

- C^0 continuity.

  That implies $P_i(1) = P_{i+1}(0)$ for all i. Therefore, we have:

$$
\sum_{k=0}^{3} y_{i+k} B_k(1) = \sum_{k=0}^{3} y_{i+1+k} B_k(0)
$$

Since that should be valid for every possible y_j values, the conditions should be on the basis functions. Therefore, we have:

$$\begin{aligned} B_0(1) &= 0 \\ B_1(1) &= B_0(0) \\ B_2(1) &= B_1(0) \\ B_3(1) &= B_2(0) \\ B_3(0) &= 0 \end{aligned}$$

- C^1 continuity.

  That implies $P_i'(1) = P_{i+1}'(0)$ for all i. Therefore, we have:

$$\sum_{k=0}^{3} y_{i+k} B_k'(1) = \sum_{k=0}^{3} y_{i+1+k} B_k'(0)$$

  As before, the conditions are:

$$\begin{aligned} B_0'(1) &= 0 \\ B_1'(1) &= B_0'(0) \\ B_2'(1) &= B_1'(0) \\ B_3'(1) &= B_2'(0) \\ B_3'(0) &= 0 \end{aligned}$$

- C^2 continuity.

  That implies $P_i''(1) = P_{i+1}''(0)$ for all i. Therefore, we have:

$$\sum_{k=0}^{3} y_{i+k} B_k''(1) = \sum_{k=0}^{3} y_{i+1+k} B_k''(0)$$

- Up to this point, we have $5 \cdot 3 = 15$ conditions. Since we have 4 basis functions each defined by a cubic polynomial, we have $4 \cdot 4 = 16$ coefficients to determine, so we need one more condition.

  In order to have the convex hull property, we need the basis functions to be non-negative and sum up to 1. Therefore, the last condition is:

$$B_0(0) + B_1(0) + B_2(0) + B_3(0) = 1$$

After solving the system of equations defined by the conditions above, we get the basis functions presented at the beginning of this section. Given the conditions that are imposed to the B-spline cubic spline, it is not surprising that the basis functions match in a very speciall way, shown in Figure 5.7.

Even though B-spline cubic splines do not interpolate the control points, they have some advantages:

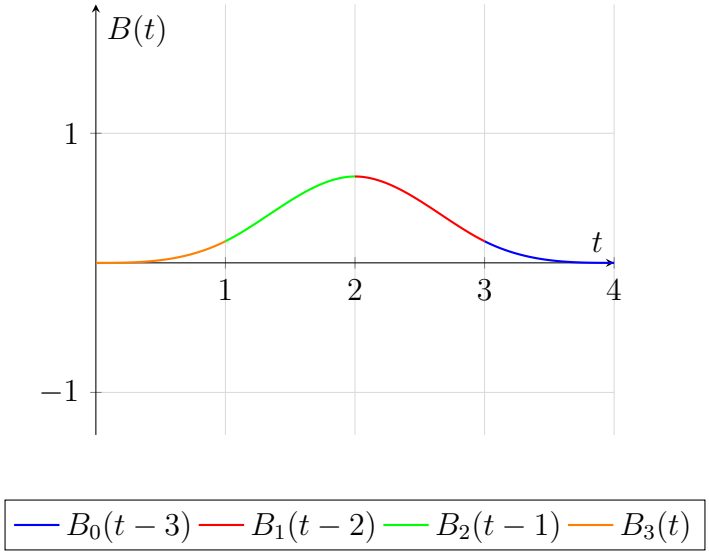


Figura 5.7: B-spline Basis Functions Match

- They do satisfy the convex hull property, as the basis functions are non-negative and sum up to 1.
- They are only controlled by points, which is more intuitive than controlling the shape of the curve by manipulating the derivatives at the endpoints of each segment, as in Hermite cubic splines.
- They do not require special care when transforming the curve, as all the control points can be transformed using the same transformation, unlike in Hermite cubic splines where we need to transform points and vectors differently.
- They achieve C^2 continuity at the knots, which is not possible with Hermite and Bezier cubic splines.

Achieving Interpolation with B-spline Cubic Splines

Even though B-spline cubic splines do not interpolate the control points, we can achieve interpolating a point by repeating it three times. Since in each segment, we have 4 control points:

- If the segments where the first three control points are the same, then the curve will start in that point.
- If the segments where the last three control points are the same, then the curve will end in that point.

Therefore, the different situations are:

- If we want to interpolate the first point, we need to repeat it three times at the beginning of the control points list, and the first segment will start in that point.
- If we want to interpolate the last point, we need to repeat it three times at the end of the control points list, and the last segment will end in that point.

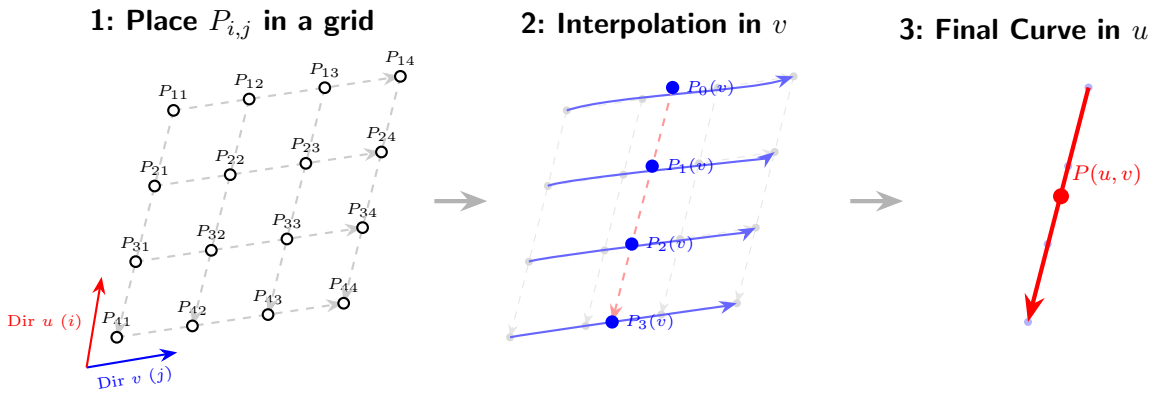


Figura 5.8: Interpolation Process on a Parametric Surface

- If we want to interpolate a point in the middle, we need to repeat it three times in the middle of the control points list, and a segment will end in that point and the next segment will start in that point.

5.3. Parametric Surfaces

Even though splines are commonly used to represent curves, they can also be used to represent surfaces. A parametric surface is a function that maps a pair of parameters (u, v) to a point in 3D space. Space is divided into patches, and each patch is defined by a parametric function. In each patch, a total of $n \cdot m$ control points are used, where:

- n is the number of control points in the u direction.
- m is the number of control points in the v direction.

After creating a grid-like structure with the control points, when interpolating a point on the surface with parameters (u, v), the process is as follows (shown in Figure 5.8):

1. First, we interpolate in the v direction to get n points.
2. Using the points obtained in the previous step as control points, we interpolate in the u direction to get the final point on the surface.

Observación. The same process can be done in the opposite order, first interpolating in the u direction and then in the v direction, and we would get the same result.

Mathematically, we can express the process as follows. Let's denote the control points as y_{ij}, where i is the index in the u direction and j is the index in the v direction. Then, we can express the interpolation process as:

$$\begin{aligned} P(u,v) &= \sum_{i=0}^{n-1} \left(\sum_{j=0}^{m-1} y_{ij} B_j(v) \right) B_i(u) \\ &= \sum_{i=0}^{n-1} \sum_{j=0}^{m-1} y_{ij} B_i(u) B_j(v) \end{aligned}$$

where $B_i(u)$ and $B_j(v)$ are the basis functions in the u and v directions, respectively. The specific type of basis functions used will depend on the type of spline we are using (e.g., Bezier, Hermite, etc.).

6. Paths and Volumes

During the previous chapters, we have been working with path tracing algorithms, but in Section 1.2 only a path was traced, which is not how light works in the real world. Physics Based Rendering (PBR) is a rendering technique that aims to simulate the behavior of light in a physically accurate way, and it is based on the Rendering Equation, which describes the equilibrium of light in a scene. In this chapter, we will firstly explore the Rendering Equation, to later understand that it does not perform well with some materials. Finally, we will adress this problem by introducing the volumetric rendering equation.

6.1. The Rendering Equation

The first step to understand the PBR is to understand the Rendering Equation (6.1), which is an integral equation that describes the equilibrium of light in a scene.

$$L_O(x, \omega, \lambda) = L_E(x, \omega, \lambda) + \int_\Omega f_r(x, \omega, \omega', \lambda) \cdot L_I(x, \omega', \lambda) \cdot (\omega' \cdot N)\ d\omega' \qquad (6.1)$$

where $L_O(x, \omega, \lambda)$ is the outgoing radiance at point x in direction ω and wavelength λ, L_E is the emitted radiance, f_r is the BRDF, L_I is the incoming radiance, N is the normal of the surface and Ω is the hemisphere of directions above the surface. The integral term represents the contribution of all the incoming light that is reflected by the surface, and it is computed by integrating over all the directions in the hemisphere.

Observación. It should be noted that $\max(0, \omega' \cdot N)$ is not needed, as the domain of integration is the hemisphere of directions above the surface, which means that $\omega' \cdot N$ is always non-negative. However, if the domain of integration were the entire sphere, then the term $\max(0, \omega' \cdot N)$ would be needed to ensure that only the directions above the surface contribute to the integral.

6.1.1. Bidirectional Reflectance Distribution Function (BRDF)

The BRDF is a function that describes how light is reflected by a surface. It is defined as the fraction of light from an incoming direction ω' that is reflected in an outgoing direction ω at a point x on the surface, for a given wavelength λ. It is denoted as $f_r(x, \omega, \omega', \lambda)$ and it has the following properties:

- $f_r(x, \omega, \omega', \lambda) \geqslant 0$: the BRDF is always non-negative.

- Helmholtz Reciprocity, which states that the BRDF is symmetric with respect to the incoming and outgoing directions. This means that the light may be inverted.

$$f_r(x, \omega, \omega', \lambda) = f_r(x, \omega', \omega, \lambda)$$

- Energy Conservation, which states that the total amount of light reflected by a surface cannot exceed the amount of light that is incident on it.

$$\int_\Omega f_r(x, \omega, \omega', \lambda) \cdot (\omega' \cdot N)\ d\omega' \leqslant 1 \qquad \forall x, \omega, \lambda$$

6.2. Participating Media

Most of the materials that we have been working with in the previous chapters are opaque, which means that they do not allow light to pass through them. However, there are many materials in the real world that are not opaque, such as smoke, fog, clouds, and even some liquids. These materials are called participating media, and they interact with light in a different way than opaque materials. There are different types of interactions that can occur when light interacts with participating media.

6.2.1. Absorption

Absorption is the process by which light is absorbed by a medium, which means that it is converted into other forms of energy, such as heat. This process can be described by the absorption coefficient $\sigma_a(x, \omega)$, where:

- x is the position in the medium.
- ω is the direction of the light.

It represents the probability of light being absorbed per unit distance traveled in the medium. The higher the absorption coefficient, the more likely it is that a photon will be absorbed as it travels through the medium. The equation that describes the absorption of light in a medium is:

$$\frac{dL(t)}{dt} = -\sigma_a(x(t), \omega) \cdot L(t) \tag{6.2}$$

where $L(t)$ is the radiance of the light at time t and $x(t)$ is the position of the light at time t. After solving this differential equation, we obtain the Beer-Lambert Law (6.3).

$$L(t) = L(0) \cdot e^{-\int_0^t \sigma_a(x(s), \omega) ds} \tag{6.3}$$

Observación. Apart from the mathematical reasoning, the Beer-Lambert Law can be intuitively understood as follows: as light travels through a medium, it has a certain probability of being absorbed at each point along its path. The longer the path, the more opportunities there are for absorption to occur, which is why the radiance decreases exponentially with distance.

6.2.2. Emission

Emission is the process by which a medium emits light, which means that it generates light on its own. This process can be described by $L_E(x, \omega)$, which represents the radiance emitted by the medium at position x in direction ω. The equation that describes the emission of light in a medium is:

$$\frac{dL(t)}{dt} = L_E(x(t), \omega)$$

where $L(t)$ is the radiance of the light at time t and $x(t)$ is the position of the light at time t. After solving this differential equation, we obtain the following equation for emission:

$$L(t) = L(0) + \int_0^t L_E(x(s), \omega) ds \tag{6.4}$$

Observación. The equation for emission can be intuitively understood as follows: as light travels through a medium, it can be emitted at any point along its path. The total radiance at time t is the initial radiance plus the contribution from all the emission that has occurred along the path up to time t.

6.2.3. Scattering

Scattering is the process by which light is scattered by a medium, which means that it changes direction as it interacts with the particles in the medium. Even though it is just one sane phenomenon, two different types of scattering are usually defined.

Out-scattering

Out-scattering is the process by which light is scattered out of its original path, which means that it is redirected in a different direction. This process can be described by the scattering coefficient $\sigma_s(x, \omega)$. The equation that describes the out-scattering of light in a medium is:

$$\frac{dL(t)}{dt} = -\sigma_s(x(t), \omega) \cdot L(t) \tag{6.5}$$

Given its similarity with the absorption equation (6.2), a new coefficient called the extinction coefficient $\sigma_t(x, \omega)$ is defined as the sum of the absorption and scattering coefficients:

$$\sigma_t(x, \omega) = \sigma_a(x, \omega) + \sigma_s(x, \omega)$$

Appart from the extinction coefficient, the *albedo* of the medium is also defined as the ratio of the scattering coefficient to the extinction coefficient:

$$\rho(x, \omega) = \frac{\sigma_s(x, \omega)}{\sigma_t(x, \omega)}$$

As with the absorption equation, after solving the out-scattering differential equation, we obtain the following generalization of the Beer-Lambert Law, which takes into account both absorption and out-scattering:

$$L(t) = L(0) \cdot e^{-\int_0^t \sigma_t(x(s), \omega) ds} \tag{6.6}$$

In-scattering

In-scattering is the process by which light is scattered into the original path, which means that it is redirected from a different direction into the original path. This process can be described by the phase function[1] $p(x, \omega, \omega')$, which represents the probability of light being scattered from direction ω' into direction ω at position x. The equation that describes the in-scattering of light in a medium is:

$$L(t) = \sigma_s(x(t), \omega) \cdot \int_{\mathbb{S}^2} p(x(t), \omega, \omega') \cdot L(t) \; d\omega' \tag{6.7}$$

Observación. The equation for in-scattering can be intuitively understood as follows: as light travels through a medium, it can be scattered into its original path from any direction. The total radiance at time t is the contribution from all the in-scattering that has occurred along the path up to time t.

6.3. The Volumetric Rendering Equation

The volumetric rendering equation is a generalization of the rendering equation that takes into account the interactions of light with participating media. The ray is parametrized, and goes from $t = s_0$ to $t = s$, where s_0 is the point where the ray enters the medium and s is the point where the ray exits the medium to reach the camera. The volumetric rendering equation is given by:

$$L(s) = L(s_0) \cdot e^{-\int_{s_0}^{s} \sigma_t(x(t), \omega) dt} + \int_{s_0}^{s} \left(L_E(x(t), \omega) \cdot e^{-\int_{t}^{s} \sigma_t(x(u), \omega) du} \right) dt \tag{6.8}$$

where $L(s)$ is the radiance at the point where the ray exits the medium (which is the point that reaches the camera) and $L(s_0)$ is the radiance at the point where the ray enters the medium. It has two terms:

- The first term represents the contribution of the radiance that enters the medium at s_0 and is attenuated by the medium as it travels to s.
- The second term represents the contribution of the radiance that is emitted by the medium along the path from s_0 to s, and is also attenuated by the medium as it travels to s.

6.4. Ray Casting Method

The volumetric rendering equation can be accurate, but has no analytical solution, which means that it cannot be solved in a closed form. Therefore, numerical methods are needed to approximate the solution. One of the most common approaches is the ray casting method, which *discretizes the path of the ray into small segments having constant opacity and emission.* The segments are usually equidistant, but they can also be adaptive.

[1] Equivalent of the BRDF.

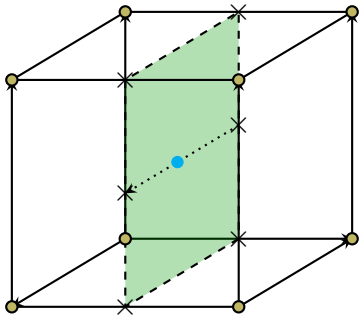


Figura 6.1: Tri-linear interpolation in a voxel grid. The blue dot represents the sample point where the interpolation is performed.

The opacity and emission of the medium are stored in a 3D grid called a *voxel grid*, which is a 3D array of voxels (volumetric pixels) that represent the properties of the medium at different points in space. In each segment of the ray, a tri-linear interpolation is performed to compute the opacity and emission of the medium at that point. As the Figure 6.1 shows, the tri-linear interpolation is performed by taking the eight vertices of the cube that contains the point where the interpolation is performed, and interpolating then in three dimensions (x, y, z) to obtain the value at the desired point.

Once the opacity and emission of the medium at each segment of the ray are computed, the color and opacity along the ray must be accumulated to obtain the final color that reaches the camera, which is called *compositing*. There are two main approaches to compositing: front-to-back and back-to-front.

6.4.1. Back-to-front compositing

In back-to-front compositing, the segments of the ray are processed from the farthest to the nearest, which means that the contribution of each segment is added to the final color after processing all the segments. The equation for back-to-front compositing in a segment with opacity α and color C is:

$$C_{\text{out}} = (1 - \alpha) \cdot C_{\text{in}} + \alpha \cdot C$$

where C_{in} is the color that has been accumulated so far, and C_{out} is the new accumulated color after processing the current segment.

6.4.2. Front-to-back compositing

In front-to-back compositing, the segments of the ray are processed from the nearest to the farthest, which means that the contribution of each segment is added to the final color as soon as it is processed. Two equations are needed for front-to-back compositing: one for the color and another for the opacity. The equations for front-to-back compositing in a segment with opacity α and color C are:

$$C_{\text{out}} = C_{\text{in}} + (1 - \alpha_{\text{in}}) \cdot \alpha \cdot C$$
$$\alpha_{\text{out}} = \alpha_{\text{in}} + (1 - \alpha_{\text{in}}) \cdot \alpha$$

where C_{in} and α_{in} are the color and opacity that have been accumulated so far, and C_{out} and α_{out} are the new accumulated color and opacity after processing the current segment.

Early ray termination

One of the advantages of front-to-back compositing is that it allows for early ray termination, which means that if the accumulated opacity reaches a certain threshold (usually close to 1), the ray can be terminated early, as it is unlikely that any further contribution will significantly affect the final color. This can significantly reduce the computational cost of rendering scenes with dense participating media, as it avoids unnecessary calculations for segments that would not contribute much to the final image.

This can't be done with back-to-front compositing, as the segments are processed in the opposite order, which means that the accumulated opacity is not available until all the segments have been processed.

Algorithm

Given the advantages of front-to-back compositing, it is the most commonly used approach in ray casting methods. In order to implement front-to-back compositing, the casted ray needs to be computed. In order to do so, two aspects are needed:

- The entry point of the ray into the volume. This can be computed representing the volume with its coordinates as `rgb` values. The final color written in the buffer represents the coordinates of the entry point of the ray into the volume.
- The direction of the ray. In order to do so, a second point is needed. There are two common approaches to compute this second point:
  - The camera position can be used as the second point, so the direction of the ray is computed as the vector from the camera position to the entry point of the ray into the volume.
  - The exit point of the ray from the volume can be computed the same way as the entry point (but only representing the back faces of the volume), and then the direction of the ray is computed as the vector from the entry point to the exit point.

Once the ray is computed, it can be discretized into segments, and the opacity and emission of the medium at each segment can be computed using tri-linear interpolation. Finally, front-to-back compositing can be performed to accumulate the color and opacity along the ray, and early ray termination can be applied when the accumulated opacity reaches a certain threshold.

6.5. Volumes Generation: Fractals

During the sections above, the reader has learnt how to render volumes, but these volumes still need to be generated. Artificial analytical volumes as spheres or

boxes can be easily generated, but they do not resemble the typical participating media that we can find in the real world, such as smoke, fog, or clouds. One of the most common approaches to generate more realistic volumes is to use fractals, which are mathematical objects that exhibit self-similarity at different scales. The fractal synthesis process is known as *Rescale-and-add*.

The rescale-and-add process is a method of generating fractals by repeatedly rescaling and adding a base image. This process creates a fractal-like structure that can produce realistic volume features. The process is iterative and involves the following steps:

1. Start with a simple base, such as simple noise or some filter.
2. Rescale the base to half the size of the original.
3. Rescale the rescaled base to half its size.
4. ...
5. The smaller the size, the more detail is added to the final volume. This process is repeated until the desired level of detail is achieved.
6. The final volume is obtained by summing all the rescaled versions, but the smaller will have more influence on the final volume than the larger ones. This is because the smaller versions add more detail to the volume, while the larger versions provide a rougher structure.

The rescale-and-add process can be implemented using the Equation (6.9), which represents the color of the volume at a given point (x, y, z) as the sum of the contributions from all the rescaled versions of the base image. The contribution of each rescaled version is weighted by a factor that decreases with each iteration, which ensures that the smaller versions have more influence on the final volume than the larger ones.

$$C(x, y, z) = \sum_{i=1}^{n} \frac{1}{2^{i-1}} \cdot F\left(\frac{1}{2^{n-i}}x, \frac{1}{2^{n-i}} \cdot y, \frac{1}{2^{n-i}} \cdot z\right) \tag{6.9}$$

The function F represents the base image, which can be generated using various different functions, such as filtered white noise, Perlin noise, images... The choice of the base image will affect the final appearance of the volume, as it determines the overall structure and features of the participating medium.

6.5.1. Parameters and their effects

In fact, the rescale-and-add process has four parameters that can be adjusted to achieve different types of volumes:

- Base function F.
- Roughness r: This parameter controls the influence of the smaller rescaled versions on the final volume.
- Octaves o: This parameter determines the number of rescaled versions that are added together to create the final volume.

- Lacunarity l: This parameter controls the rescaling factor between successive octaves.

Using those parameters, we can create a wide variety of volumes. The Equation (6.10) represents the color of the volume at a given point (x, y, z) with the parameters included:

$$C(x, y, z) = \sum_{i=1}^{o} \frac{1}{2} \cdot r^{i-1} \cdot F\left(\frac{1}{l^{o-i}}x, \frac{1}{l^{o-i}}y, \frac{1}{l^{o-i}}z\right) \tag{6.10}$$

We can observe that the Equation (6.9) is a special case of the Equation (6.10) when $r = 0{,}5$, $o = n$ and $l = 2$.

6.5.2. Multifractals

Multifractals are a generalization of fractals that can model more complex and heterogeneous structures. In the context of volume synthesis, multifractals can be used to create volumes with varying degrees of roughness and detail across different regions. In order to achieve this, we can have different base functions $F_i(x, y, z)$, each one with its own weight $a_i(x, y, z)$, which determines the influence of each base function on each point of the volume. The functions a_i have to satisfy the following conditions forall point (x, y, z):

1. $a_i(x, y, z) \geqslant 0 \qquad \forall i$, which means that the weights must be non-negative.
2. $\sum_i a_i(x, y, z) = 1$, which means that the weights must sum to 1 at each point.

Under these conditions, the function F of the Equation (6.10) can be defined as a weighted sum of the base functions:

$$F(x, y, z) = \sum_i a_i(x, y, z) \cdot F_i(x, y, z) \tag{6.11}$$

This allows for the creation of different types of volumes in different regions, as the weights can be adjusted to give more influence to certain base functions in specific areas.

7. Radiosity

In order to achieve global illumination, the Rendering Equation (Equation 6.1) must be solved. Three main approaches exist for solving it:

- Path tracing: it is a Monte Carlo method that estimates the solution of the rendering equation by randomly sampling light paths. It was explained in Section 1.2.
- Radiosity: it will be explained in detail in this chapter.
- Photon mapping: it will be explained in detail in Chapter 9.

In order to understand the Radiosity method, some radiometric quantities need to be defined first. Radiometry is the science of measuring electromagnetic radiation, including visible light. It provides a set of quantities that are used to describe the properties of light and how it interacts with surfaces.

Definición 7.1 (Radiant Energy)**.** Radiant energy is the total amount of energy that is carried by a beam of light.

$$Q \qquad [J]$$

Definición 7.2 (Radiant Flux)**.** Also known as Radiant Power, it is the rate at which radiant energy is transferred or emitted. It is defined as the amount of radiant energy per unit time.

$$\Phi = \frac{dQ}{dt} \qquad [W = {}^{J}/_{s}]$$

Definición 7.3 (Irradiance)**.** Irradiance is the amount of radiant flux that is incident on a surface per unit area.

$$E = \frac{d\Phi}{dA} \qquad [{}^{W}/_{m^2}]$$

Definición 7.4 (Radiosity)**.** Radiosity is the amount of radiant flux that is outgoing from a surface per unit area.

$$B = \frac{d\Phi}{dA} \qquad [{}^{W}/_{m^2}]$$

Observación. The difference between irradiance and radiosity is that the former measures the incoming light on a surface, while the latter measures the outgoing light from a surface. In other words, irradiance is the amount of light that hits a surface, while radiosity is the amount of light that leaves a surface.

For the next measures, the solid angle is needed. It is a measure of the amount of space that an object occupies in the field of view of an observer. It is defined as the area of the projection of the object onto a unit sphere centered at the observer. More generally, if the sphere used for the projection has a radius r, and the surface area of the projection is A, the solid angle Ω is defined as:

$$\Omega = \frac{A}{r^2} \qquad [sr]$$

The unit of solid angle is the steradian (sr) (an adimensional unit), which is defined as the solid angle subtended by an area of r^2 on the surface of a sphere of radius r. It is the three-dimensional equivalent of the two-dimensional radian.

Definición 7.5 (Radiant Intensity)**.** Radiant intensity is the amount of radiant flux that is emitted by a source in a particular direction per unit solid angle.

$$I = \frac{d\Phi}{d\overrightarrow{\omega}} \qquad [W/sr]$$

Definición 7.6 (Radiance)**.** Radiance is the amount of radiant flux that is emitted, reflected, transmitted or received by a surface per *projected* unit area and per unit solid angle.

$$L = \frac{dI}{dA \cdot \cos(\theta)} = \frac{d^2\Phi}{dA \cdot \cos(\theta) \cdot d\overrightarrow{\omega}} \qquad [W/m^2 \cdot sr]$$

where θ is the angle between the normal of the surface and the direction of the incoming light. The term $\cos(\theta)$ is used to account for the fact that the effective area of the surface that is illuminated by the light decreases as the angle of incidence increases.

Radiance is the most used quantity in computer graphics, because it is the one that is conserved[1] along a ray of light. This means that the radiance of a ray of light does not change as it travels through space, unless it interacts with a surface. This property makes it easier to compute the illumination of a scene, because we can trace rays of light from the camera to the scene and compute the radiance at each point of intersection.

Observación. So many measures may be confusing. The basic one is the radiant energy, and then the radiant flux is the rate of change of the radiant energy. All of the other measures are derived from the flux energy connecting it with other quantities:

- Irradiance and Radiosity: Flux per unit area.
- Radiant Intensity: Flux per unit solid angle.
- Radiance: Flux per projected unit area and per unit solid angle.

[1] In fact, it only happens in vacuum.

Two essential connections between Radiance and Irradiance and between Radiance and Radiosity are given by the following formulas:

$$E(x) = \int_{\Omega} L_i(x, \overrightarrow{\omega}) \cdot \cos(\theta) \cdot d\overrightarrow{\omega} \tag{7.1}$$

$$B(x) = \int_{\Omega} L_o(x, \overrightarrow{\omega}) \cdot d\overrightarrow{\omega} \tag{7.2}$$

where Ω is the hemisphere of directions above the point x, and θ is the angle between the normal of the surface at point x and the direction $\overrightarrow{\omega}$.

Definición 7.7 (Reflectance)**.** Reflectance is the fraction of incident light that is scattered when it hits a surface (that is not absorbed).

$$\rho(x) = \frac{\text{Reflected Flux}}{\text{Incident Flux}} \qquad \text{[adimensional]}$$

This last material property connects the irradiance and the radiosity of a surface with the following formula:

$$B(x) = \rho(x) \cdot E(x) \tag{7.3}$$

Once all these concepts have been defined, the Radiosity method can be explained.

7.1. The Radiosity Equation

In order to solve the rendering equation using the Radiosity method, it needs to be translated into radiosity terms. Three first changes are needed:

- Iterating over points instead of directions:

  Instead of iterating over all the possible $\overrightarrow{\omega}'$ directions of the Ω hemisphere above the surface, it iterates over all the points y in the scene S. These points are infinitelly small with area dA_y.

- Visibility factor:

  Since it iterates over all the points y in the scene, some of them will not be visible from point x (that is, there will be an object between them). Therefore, only the points y that are visible from x should be considered. In order to do that, a visibility factor $V(x, y)$ is included, which is 1 if the points x and y are visible to each other (that is, there is no object between them), and 0 otherwise.

  $$V(x, y) = \begin{cases} 1 & \text{if } x \text{ and } y \text{ are visible to each other} \\ 0 & \text{otherwise} \end{cases}$$

  Observación. Since we consider points y with an infinitesimal area, the visibility factor is either 0 or 1. However, if we consider patches with a finite area, the visibility factor can take any value between 0 and 1, depending on the fraction of the patch that is visible from point x.

- Geometry factor:

  A change of variable from ω' to A_y is needed. From the definition of solid angle, we have that:

  $$\omega' = \frac{A_{y,\text{projected}}}{r^2} \Longrightarrow d\omega' = \frac{dA_{y,\text{projected}}}{r^2}$$

  where $r = \|x - y\|$ is the distance between points x and y, and $A_{y,\text{projected}}$ is the area of the projection of the patch at point y onto a plane tangent to the sphere of radius r centered at point x used for the solid angle. That projection is given by:

  $$A_{y,\text{projected}} = A_y \cdot \cos(\theta_y)$$

  where θ_y is the angle between the normal of the surface at point y (N') and the vector from y to x, which is $-\omega'$. Therefore, we have that:

  $$A_{y,\text{projected}} = A_y \cdot \cos(\theta_y) = A_y \cdot (N' \cdot (-\omega'))$$

  Taking this into account, we have:

  $$d\omega' = \frac{dA_{y,\text{projected}}}{r^2} = \frac{dA_y \cdot (N' \cdot (-\omega'))}{\|y - x\|^2}$$

  In order to have a simpler formula, a geometry factor $G(x, y)$ is defined so that:

  $$G(x, y)dA_y = (\omega' \cdot N)\ d\omega' \Longrightarrow G(x, y) = \frac{(\omega' \cdot N) \cdot (N' \cdot (-\omega'))}{\|y - x\|^2}$$

  This geometry factor accounts for the orientation of the surfaces at points x and y, as well as the distance between them.

Taking these three changes into account, the rendering equation can be rewritten[2] as:

$$L(x, \lambda, \omega) = L_e(x, \lambda, \omega) + \int_{y \in S} f_r(x, \lambda, \omega', \omega) \cdot L_i(y, \lambda, \omega') \cdot V(x, y) \cdot G(x, y)\ dA_y$$

7.1.1. Diffuse Reflection Simplification

The radiosity method is based on the assumption that all surfaces in the scene are perfectly diffuse reflectors, which means that they reflect light equally in all directions. This simplification allows to further simplify the rendering equation, as the BRDF of a perfectly diffuse reflector is constant and equal to:

$$f_r(x, \lambda, \omega', \omega) = \frac{\rho(x)}{\pi}$$

[2]It should be noted that it is simply rewritten, not approximated.

where $\rho(x)$ is the reflectance of the surface at point x. Therefore, the rendering equation can be rewritten as:

$$L(x, \lambda, \omega) = L_e(x, \lambda, \omega) + \int_{y \in S} \frac{\rho(x)}{\pi} \cdot L_i(y, \lambda, \omega') \cdot V(x, y) \cdot G(x, y) \; dA_y$$

In order to simplify the notation even more, a *transfer function* $G'(x, y)$ is defined as:

$$G'(x, y) = \frac{V(x, y) \cdot G(x, y)}{\pi}$$

so the rendering equation can be rewritten as:

$$L(x, \lambda, \omega) = L_e(x, \lambda, \omega) + \rho(x) \cdot \int_{y \in S} L_i(y, \lambda, \omega') \cdot G'(x, y) \; dA_y$$

7.1.2. Translating Radiance to Radiosity

Changing the perspective from radiance to radiosity, the resulting equation is:

$$B(x) = B_e(x) + \rho(x) \cdot \int_{y \in S} B(y) \cdot G'(x, y) \; dA_y$$

where $B_e(x)$ is the emitted radiosity at point x.

7.1.3. The Discretized Radiosity Equation

In order to solve the radiosity equation, the scene is divided into small patches with:

- Finite area A_i.
- Constant radiosity B_i.
- Constant reflectance ρ_i.
- Constant emitted radiosity B_i^e.

In addition, the integral of the geometry factor is approximated by a form factor F_{ij}, which is defined as:

$$F_{ij} = \frac{1}{A_i} \int_{x \in A_i} \int_{y \in A_j} G'(x, y) \; dA_y \; dA_x$$

This form factor represents the fraction of the radiosity that leaves patch i and arrives at patch j. Therefore, the radiosity equation can be rewritten as:

$$B_i = B_i^e + \rho_i \cdot \sum_{j=1}^{N} B_j \cdot F_{ij} \tag{7.4}$$

where N is the total number of patches in the scene.

The "direction" of the Form Factors

The reader could think that, if F_{ij} represents the fraction of the radiosity that leaves patch i and arrives at patch j, then the Equation 7.4 should use F_{ji} instead of F_{ij}. In order to understand why F_{ij} is used, the following connection between both form factors is needed:

$$A_i \cdot F_{ij} = A_j \cdot F_{ji}$$

From the Equation 7.4, if every term is multiplied by A_i (representing the total radiant flux leaving patch i), we have:

$$A_i \cdot B_i = A_i \cdot B_i^e + \rho_i \cdot \sum_{j=1}^{N} B_j \cdot A_i \cdot F_{ij}$$

If the connection between the form factors is applied, we have:

$$A_i \cdot B_i = A_i \cdot B_i^e + \rho_i \cdot \sum_{j=1}^{N} B_j \cdot A_j \cdot F_{ji}$$

which is the reasonable interpretation of the equation: the total radiant flux leaving patch i is equal to the emitted radiant flux from patch i plus the reflected radiant flux from patch i, which is the portion of the radiant flux that arrives at patch i from all the other patches j (which is $B_j \cdot A_j \cdot F_{ji}$) multiplied by the reflectance ρ_i. Therefore, when working with radiant flux, the form factor F_{ji} may be used having a "reasonable" interpretation of the equation, but when working with radiosity, the form factor F_{ij} is needed.

Obtaining the Form Factors

The definition of the form factors was given by:

$$F_{ij} = \frac{1}{A_i} \int_{x \in A_i} \int_{y \in A_j} G'(x, y) \; dA_y \; dA_x$$

This is the most computationally expensive part of the radiosity method, because $G'(x, y)$ is a complex function that depends on the geometry of the scene and the visibility between patches i and j. Therefore, several methods have been proposed to approximate the form factors.

- Hemicube method: it is a method that aproximates the foam factor between a differential patch dA_i and a patch A_j by projecting the patch A_j onto a hemispherical sphere centered at the point x of the patch dA_i.
  - The projection onto the sphere takes into account $N' \cdot (-\omega')$ and $\|y - x\|^2$.
  - The projection onto the circle takes into account $N \cdot \omega'$ and π.

  However, projecting the patch A_j onto a hemispherical sphere is computationally expensive. Therefore, the hemicube method approximates the projection onto the hemisphere by projecting the patch A_j onto a hemicube centered at the point x of the patch dA_i. The projection onto the hemicube is much faster than the projection onto the hemisphere, as it can be implemented rasterizing from x using 5 different viewing directions (one for each face of the hemicube).

7.2. The Radiosity Method

Once the radiosity equation is obtained, let's explain how everythig is combined in order to obtain the final image.

1. The scene is divided into small patches with finite properties and constant properties.

2. The form factors between all the patches are calculated using one of the different methods that exist for that purpose, as the hemicube method. It is the most computationally expensive part of the radiosity method, as obtaining the visibility between patches and the geometry factor is complex.

   It should be noted that the form factors only depend on the geometry of the scene and the visibility between patches, but not on the material properties of the patches. Therefore, if the geometry of the scene does not change, the form factors can be precomputed and reused for different material properties.

3. For each patch and each component of the light[3], the radiosity for that patch and that component of the light is calculated. Writing the Radiosity Equation (Equation 7.4) in matrix form, we have:

$$\begin{pmatrix} B_1 \\ B_2 \\ \vdots \\ B_N \end{pmatrix} = \begin{pmatrix} B_1^e \\ B_2^e \\ \vdots \\ B_N^e \end{pmatrix} + \begin{pmatrix} \rho_1 0 & \cdots & 0 & \\ 0 & \rho_2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \rho_N \end{pmatrix} \begin{pmatrix} F_{11} & F_{12} & \cdots & F_{1N} \\ F_{21} & F_{22} & \cdots & F_{2N} \\ \vdots & \vdots & \ddots & \vdots \\ F_{N1} & F_{N2} & \cdots & F_{NN} \end{pmatrix} \begin{pmatrix} B_1 \\ B_2 \\ \vdots \\ B_N \end{pmatrix}$$

   Defining a matrix T called the transfer matrix as:

$$T = \begin{pmatrix} \rho_1 0 & \cdots & 0 & \\ 0 & \rho_2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \rho_N \end{pmatrix} \begin{pmatrix} F_{11} & F_{12} & \cdots & F_{1N} \\ F_{21} & F_{22} & \cdots & F_{2N} \\ \vdots & \vdots & \ddots & \vdots \\ F_{N1} & F_{N2} & \cdots & F_{NN} \end{pmatrix}$$

   the radiosity equation can be rewritten as:

$$B = B^e + T \cdot B$$

   There are two main methods for solving this equation and obtaining the radiosity of each patch:

   - Algebraic method: it consists in rearranging the equation as:

$$\begin{aligned} B &= B^e + T \cdot B \\ (I - T) \cdot B &= B^e \\ B &= (I - T)^{-1} \cdot B^e \end{aligned}$$

     where I is the identity matrix. However, this method is computationally expensive, as it requires inverting the matrix $I-T$, which has a complexity of $O(N^3)$, where N is the number of patches in the scene.

[3] In practice, the light is usually divided into three components: red, green and blue. However, theoretically, it could be computed for each wavelength of light.

- Iterative method: it consists in iteratively applying the equation until convergence. The equation can be solved iteratively as:

$$B^{(0)} = B^e$$
$$B^{(k+1)} = E + T \cdot B^{(k)}$$

where $B^{(k)}$ is the radiosity at iteration k, representing the light until k bounces.

This method is more efficient than the algebraic method, as it has a complexity of $O(N^2)$ per iteration, and it usually converges in a few iterations.

4. Once the radiosity of each patch is obtained, a radiosity value needs to be assigned to each vertex of the scene, as the final image is rendered using rasterization. There are two main methods for doing that:

   - Flat Method: it consists in assigning the radiosity of the patch to all the vertices of the patch. This method is simple, but it can produce artifacts in the edge of the patches, as the radiosity changes abruptly between patches.
   - Smooth Method: it consists in assigning the radiosity of each patch to the vertices of the patch, and then averaging the radiosity of the vertices that belong to multiple patches. This method produces smoother results, as it avoids abrupt changes in the radiosity between patches.

     However, it is more computationally expensive than the flat method, and it needs to take into account the normals, as there are cases where real edges exist in the scene.

5. Finally, the final color for the corresponding light component of each vertex is calculated from the radiosity as follows:

$$C = \frac{B}{B+1}$$

This formula is used to convert the radiosity, which can take any value in $[0, +\infty[$, to a color value in $[0, 1[$. The formula is designed so that when the radiosity is low, the color is close to 0, and when the radiosity is high, the color approaches 1.

6. Once the color of each vertex is obtained, the final image can be rendered using rasterization.

8. Irradiance Volumes

Irradiance volumes are a technique used in computer graphics to approximate the global illumination of a scene. It works as follows:

1. The scene is divided into a grid of cells (using spatial partitioning techniques such as octrees).
2. For each vertex of those cells, the irradiance in every direction is computed and stored in a data structure (e.g., spherical harmonics).
3. During rendering, the irradiance at a point in the scene is approximated by interpolating the irradiance values from the surrounding cells. Once the irradiance is known, using the Equation 7.3, the radiosity can be computed and used to shade the point (as explained in Chapter 7).

This technique is only useful under certain assumptions:

- The object being shaded is affected by the scene's indirect lighting, but it does not affect the scene.
- The scene (apart from the object being shaded) is static, so the irradiance values can be precomputed and reused.
- The object being shaded is a diffuse surface (we will understand why).

8.1. Generating the Irradiance Volume

In order to generate the irradiance volume, the irradiance at each vertex of the grid must be computed. Taking the Equation 7.1 into account, the irradiance at a point x can be computed by integrating the incoming radiance over the hemisphere of directions above the point x; depending on the normal of the surface at point x. Therefore, the irradiance must be computed for every possible normal direction, so each vertex of the grid must store the irradiance as a spherical function.

For each vertex of the grid, the following steps are performed in order to compute the irradiance:

1. Only a finite set of directions over the sphere $\{\omega_1, \omega_2, \ldots, \omega_n\}$ is considered. The incoming radiance for those normal directions is computed by tracing rays from the vertex in those directions and computing the radiance that arrives (using ray tracing or other techniques). In this step, the assumption that the object being shaded is a diffuse surface is important, as it allows us to simplify its BRDF to a constant value.

2. Once the incoming radiance is computed, the irradiance can be computed using the Equation 7.1, which can be approximated by a discrete sum:

$$E(x, N) \approx \sum_{i=1}^{n} L_i(x, \omega_i) \cdot \text{máx}\{0, \omega_i \cdot N\} \cdot \Delta\omega_i$$

where N is the normal of the surface at point x, and $\Delta\omega_i$ is the solid angle associated with the direction ω_i.

This process is repeated for every vertex of the grid, and the resulting spherical function of irradiance is stored in a data structure that allows for efficient interpolation during rendering.

8.2. Interpolating the Irradiance Volume

During rendering, the irradiance at a point in the scene is approximated by interpolating the irradiance values from the surrounding cells. Two main problems arise when interpolating the irradiance values:

1. If the grid was not uniform, the interpolation must take into account the different sizes of the cells. For instance, if the point is inside a really large cell but is next to a vertex of a smaller neighboring cell, the irradiance value of that vertex should be taken into account. In order to solve this problem, two different approaches can be used:
   - Instead of using trilinear interpolation, a weighted and more sophisticated interpolation method can be used.
   - Normal trilinear interpolation can be used, but the values of the vertex of subsequent smaller cells must match the corresponding interpolated irradiance value from the larger cell.
2. The irradiance values are stored as spherical functions, so the interpolation must take into account this spherical nature, as otherwise the interpolated result is the opposite of what is expected. In order to solve this, the irradiance value is not usually interpolated directly, but the gradients of the irradiance are interpolated instead.

9. Photon Mapping

As introduced in Chapter 7, in this chapter we will cover photon mapping. One specific visual effect that other rendering techniques struggle to achieve is caustics, and photon mapping renders them efficiently.

Definición 9.1 (Caustics)**.** Caustics are the patterns of light that are formed when light rays are reflected or refracted by a curved surface or object. They are small bright regions of light that are concentrated in certain areas, creating a visually striking effect. These patterns can be seen in everyday life, such as:

- The bright patterns of light at the bottom of a swimming pool on a sunny day.
- The focused light patterns created by a glass of water or a crystal.
- The shimmering light patterns on the surface of a body of water, such as a lake or ocean.

Caustics are rare but high-energy events, and they are difficult to render with traditional ray tracing techniques. Photon mapping is a two-pass global illumination algorithm that can efficiently simulate caustics and other complex lighting effects. It consists of two phases:

1. Photon Emission: In this phase, photons are emitted from light sources and traced through the scene.
2. Rendering: In this phase, the scene is rendered using the information from the first pass to compute the final image.

The information gathered during the photon emission phase is stored in a data structure called a *photon map*.

9.1. Photon Emission Phase

During the photon emission phase, photons are emitted from light sources and traced through the scene. The emission depends on the type of light source being used or the scene being rendered, but a total of N photons are emitted. Each photon always consists of:

- A position in 3D space. Initially, the position of the photon is the position of the light source.

- A direction in 3D space. The initial direction of the photon is determined by the type of light source being used, and will change as the photon interacts with surfaces in the scene.

- A power value on each of the RGB channels. The power of the photon is determined by the total power of the light source and the number of photons being emitted. The initial power of each photon is given by:

$$\Phi_{\text{photon}} = \frac{\Phi_{\text{light source}}}{N}$$

With each interaction with a surface, the photon interacts with the surface and the intersection is stored in the photon map. When the photon interacts with the surface, it can result in:

- Absorption: The power of the photon is atenuated by the absorption coefficient of the surface (with a different absorption coefficient for each RGB channel), but the photon still continues to be traced through the scene.

- Diffusion: The photon is scattered in a random direction, depending on the properties of the surface. In addition, the photon is stored in the photon map.

- Specular/Refraction: The photon is reflected or refracted in a specific direction, depending on the properties of the surface and the angle of incidence.

In order to understand what happens to the photon, two previous concepts are used:

- The *BSDF* (Bidirectional Scattering Distribution Function) is a function that describes how light is scattered by a surface. It is used to calculate the probability of each type of interaction (reflection, refraction, diffusion, or absorption) based on the material properties of the surface.

- The *Russian roulette* technique is a probabilistic method that consists of randomly choosing one of the possible interactions based on the probabilities calculated from the BSDF.

When a photon intersects with a surface, the BSDF is used to determine the probabilities of reflection, refraction, diffusion, or absorption. A random number is generated, and based on the probabilities, one of the interactions is chosen using the Russian roulette technique. In addition, the following information is stored in the photon map for each photon that intersects with a diffuse surface:

- The position of the intersection.

- The incoming direction of the photon.

- The normal of the surface at the intersection.

- The power of the photon at the intersection.

There are different data structures that can be used to store the photon map in order to optimize the search for nearby photons during the rendering phase, such as those seen in Section 2.3.2. The most common data structure used is the *kd tree*. Two different photon maps are usually created:

- Caustic Photon Map: This photon map stores only the photons that have been reflected or refracted by specular surfaces, and is used to render caustics.

  $$\text{Light} \rightarrow \text{Specular Chain} \rightarrow \text{Diffuse Surface} \rightarrow \text{Eye}$$

- Global Photon Map: This photon map stores all the other intersections of photons with surfaces, and is used to render the global illumination of the scene.

When photons are traced, they are either classified as caustic photons or global photons, depending on the type of the first specular surface they interact with. If a photon interacts with a diffuse surface before interacting with any specular surfaces, it is classified as a global photon. If a photon interacts with a specular surface before interacting with any diffuse surfaces, it is classified as a caustic photon.

9.2. Rendering Phase

During the rendering phase, the scene is rendered using the information stored in the photon map. It approximates the Rendering Equation (Equation 6.1) by the following equation:

$$L_o(x,\omega,\lambda) = L_e(x,\omega,\lambda) + L_o^{\text{direct}}(x,\omega,\lambda) + L_o^{\text{indirect}}(x,\omega,\lambda) + L_o^{\text{caustics}}(x,\omega,\lambda) + L_o^{\text{specular}}(x,\omega,\lambda)$$

Let's break down each term in the equation:

- $L_o(x,\omega,\lambda)$: The outgoing radiance at point x in direction ω for wavelength λ.
- L_e: The emitted radiance. This term accounts for any light that is emitted directly from the surface itself, such as light sources or emissive materials.
- L_o^{direct}: The direct illumination represents the contribution via direct illumination by the light sources. It is calculated using shadow rays and Phong illumination.
- L_o^{indirect}: The indirect illumination represents the contribution via indirect illumination by other surfaces in the scene. Instead of directly sampling the global photon map at the point of intersection, the indirect illumination is computed by a technique called *final gathering*, that consists approximating the integral in the Rendering Equation using Monte Carlo integration. This involves shooting rays from the point of intersection in random directions and sampling the global photon map in each of these rays to estimate the contribution of indirect illumination. The contributions from all the sampled rays are then averaged to obtain an estimate of L_o^{indirect}.

- L_o^{caustics}: The caustics term represents the contribution of light that form caustics in the scene. It is computed by sampling the caustic photon map at the point of intersection and estimating the contribution of caustics to the outgoing radiance.

  Observación. Since the caustic photon map is directly seen, it needs to have a high density of photons. The global photon map, on the other hand, is only indirectly seen (final gathering), so it can have a lower density of photons. This is because the final gathering technique averages the contributions from multiple rays, which helps to smooth out the noise in the estimate of L_o^{indirect}.

- L_o^{specular}: The specular term represents the contribution of light that is reflected or refracted by specular surfaces in the scene. It is computed by tracing rays from the point of intersection in the direction of reflection or refraction, and recursively computing the outgoing radiance at the new intersection points.

When a photon map is sampled at a point x in order to obtain the outgoing radiance, the following approximation of the Rendering Equation (Equation 6.1) is used:

$$L_o(x,\omega,\lambda) = \int_\Omega f_r(x,\omega',\omega,\lambda) L_i(x,\omega',\lambda)(\omega' \cdot n) d\omega' \approx \frac{1}{\pi r^2} \sum_{p \in N(x)} f_r(x,\omega_p,\omega,\lambda)\Phi_p(\lambda)$$

Where:

- $N(x)$: The set of photons in the photon map that are considered to be near the point x. They are all contained within a sphere of radius r centered at x[1].

- $\Phi_p(\lambda)$: The power of the photon p at wavelength λ.

- ω_p: The incoming direction of the photon p.

A key aspect in this approximation is $N(x)$, which is the set of photons that are considered to be near the point x. There are two main approaches:

- Fixed radius: In this approach, a fixed radius r is used to define the sphere centered at x. All photons within this sphere are considered to be near the point x and are included in the set $N(x)$, so the number of photons in $N(x)$ can vary depending on the density of photons in the scene.

  - If the radius is too small, there may not be enough photons in $N(x)$ to accurately estimate the outgoing radiance, resulting in noise and artifacts in the final image.
  - If the radius is too large, the estimate of the outgoing radiance may be biased, as it may include photons that are not actually contributing to the illumination at the point x.

[1] It is divided by πr^2 because, even though they are contained in a sphere, the photons are assumed to be uniformly distributed over a disk of radius r centered at x (with area πr^2).

- Fixed number of photons: In this approach, a fixed number of photons k is used to define the set $N(x)$. The k nearest photons to the point x are included in the set $N(x)$, so the radius of the sphere centered at x can vary depending on the density of photons in the scene. The selection of the hyperparameter k is important, as it can affect the accuracy and efficiency of the photon mapping algorithm.

9.3. Bias and Consistency

An important aspect of photon mapping is that it is a biased rendering algorithm. This means that averaging infinitely many renders of the same scene using this method does not converge to a correct solution to the rendering equation. However, it is a consistent method, and the accuracy of a render can be increased by increasing the number of photons. As the number of photons approaches infinity, a render will get closer and closer to the solution of the rendering equation.

Given that photon mapping is a biased rendering algorithm, the rare light paths are taken into account, which could be missed by other unbiased rendering algorithms.

9.4. Common Artifacts

Three common artifacts that can occur in photon mapping are:

- Blotches: Blotches are caused by a low density of photons in the photon map, which can result in areas of the image that appear darker or lighter than they should be. This can be mitigated by increasing the number of photons emitted during the photon emission phase.
- Leaks: Leaks occur when photons are taken into account through opace surfaces when sampling the photon map, which can result in light bleeding through surfaces that should be opaque. This can be mitigated by applying filters to the photon map to remove photons that are not contributing to the illumination at a given point.

9.5. Progressive Photon Mapping

Progressive photon mapping is a variant of the photon mapping algorithm that addresses some of the limitations of the original algorithm. It is an iterative algorithm that progressively refines the photon map and the final image over multiple passes. In each pass, a fixed number of photons are emitted and traced through the scene, and the photon map is updated with the new information. The radius used to define the set of nearby photons $N(x)$ is also progressively reduced over multiple passes, which allows for a more accurate estimate of the outgoing radiance at each point in the scene. This approach can help to reduce artifacts such as blotches and leaks, and can also improve the overall quality of the final image.

10. Precomputed Radiance Transfer (PRT)

In previous chapters, we have seen different methods for solving the Rendering Equation in order to compute global illumination in a scene. In this chapter, we will introduce a method that allows us to solve the Rendering Equation in real-time, but with some limitations. This method is called Precomputed Radiance Transfer (PRT). The main idea behind PRT is to precompute as much of the lighting information as possible, so that at runtime we can quickly evaluate the lighting for a given scene.

10.1. Assumptions and Simplifications

To make the problem of solving the Rendering Equation tractable, we need to make some assumptions and simplifications. The three main simplifications we will make are:

- Diffuse surfaces

  A diffuse surface is one that scatters light equally in all directions. This means that the BRDF for a diffuse surface is constant and does not depend on the incoming or outgoing directions. This simplification allows us to ignore the directional dependence of the BRDF, which makes the problem easier to solve. It is defined by the material's albedo ρ, which is the fraction of incoming light that is reflected by the surface. The BRDF for a diffuse surface can be expressed as:

  $$f_r(x, \omega, \omega', \lambda) = \frac{\rho}{\pi}$$

  where the factor of π is included to ensure energy conservation, as the reflected light is distributed uniformly over the hemisphere.

- Only first van Neumann Term

  The Rendering Equation itself hides recursive terms, as the incoming radiance L_i can be expressed is itself the outgoing radiance from another point in the scene. This means that to solve the equation, we would need to evaluate it recursively. Each level in the recursion is called a van Neumann term, and represents the contribution of light that has bounced a certain number of times in the scene. The first van Neumann term represents the direct lighting, which is the contribution of light that comes directly from the light sources to the point being shaded. By only considering the first van Neumann term, we

are ignoring any indirect lighting, which is the contribution of light that has bounced off other surfaces in the scene before reaching the point being shaded.

- Distant Lighting

  Distant lighting is a simplification that assumes that the light sources in the scene are infinitely far away. This means that the incoming light can be treated as parallel rays, and the direction of the incoming light does not change across the scene.

  This simplification, along with the previous one, allows us to simplify the incoming radiance term in the Rendering Equation as:

$$L_i(x, \omega', \lambda) = L_{\text{env}}(\omega') \cdot V(x, \omega')$$

  where $L_{\text{env}}(\omega')$ is the radiance coming from the environment in the direction ω', and $V(x, \omega')$ is a visibility term represents whether the point x is visible from the direction ω'.

  In addition, no emission is considered, so the term $L_e(x, \omega, \lambda)$ is ignored.

Therefore, with these simplifications, the Rendering Equation can be rewritten as the Equation (10.1):

$$L_o(x, \omega, \lambda) = \frac{\rho}{\pi} \int_\Omega L_{\text{env}}(\omega') \cdot V(x, \omega') \cdot (\omega' \cdot N) \; d\omega' \tag{10.1}$$

The term $V(x, \omega') \cdot (\omega' \cdot N)$ is called the transfer function, and it represents how much of the incoming light from the environment is transferred to the outgoing radiance at the point x in the direction ω.

$$T(x, \omega') = V(x, \omega') \cdot \text{máx}(0, \omega' \cdot N)$$

10.2. Converting the Integral into a Dot Product

The idea that PRT introduces is project the possible integrals onto a basis, so that we can simply convert the integral into a dot product. Let's consider the space of all the possible functions with domain A and range B. Given that this space is infinite-dimensional, every basis will have an infinite number of elements. Chosen a basis $\{b_i\}$, we can express any function $f : A \to B$ as a linear combination of the basis elements:

$$f(x) = \sum_{i=0}^{\infty} c_i b_i(x)$$

where c_i are the coefficients of the linear combination:

$$c_i = \langle f(x), b_i(x) \rangle = \int_A f(x) b_i(x) \; dx$$

For the PRT to really simplify the problem, we need to choose an specific type of basis, called an orthogonal basis.

Definición 10.1 (Orthogonal Basis)**.** A basis $\{b_i\}$ is orthogonal if for any two different elements b_i and b_j, the inner product $\langle b_i, b_j \rangle$ is zero. This means:

$$\langle b_i(x), b_j(x) \rangle = \int_A b_i(x) b_j(x)\ dx = \delta_{ij} = \begin{cases} 0 & \text{if } i \neq j \\ 1 & \text{if } i = j \end{cases}$$

In order to understand the idea that lies behind PRT, let's consider two functions f and g, and an orthogonal basis $\{b_i\}$. Given that we are only interested in approximating the integral of the product of f and g, only the first N coefficients of the linear combination will be relevant. Let's then assume that the projection of f, g onto the basis $\{b_i\}$ is given by:

$$f(x) \approx \sum_{i=0}^{N} c_i b_i(x) \qquad g(x) \approx \sum_{i=0}^{N} d_i b_i(x)$$

Then, the integral of the product of f and g can be approximated as:

$$\int_A f(x)g(x)\ dx \approx \int_A \left(\sum_{i=0}^{N} c_i b_i(x) \right) \left(\sum_{j=0}^{N} d_j b_j(x) \right)\ dx = \sum_{i=0}^{N} \sum_{j=0}^{N} c_i d_j \int_A b_i(x) b_j(x)\ dx \overset{(*)}{=}$$
$$\overset{(*)}{=} \sum_{i=0}^{N} c_i d_i \int_A b_i(x) b_i(x)\ dx = \sum_{i=0}^{N} c_i d_i$$

where in $(*)$ we used the fact that the basis is orthogonal. Therefore, we have approximated the integral of the product of f and g as a dot product of their coefficients in the ortogonal basis.

10.2.1. Spherical harmonics (SH)

The choice of the basis is crucial for the success of PRT. The most common choice for the basis in PRT is the Spherical Harmonics (SH). Spherical Harmonics are a set of *orthogonal* functions defined on the surface of a unit sphere and with possible complex values. However, and given that we are only using real functions, we will only consider the real spherical harmonics. The fact that they are defined on the surface of a unit sphere makes them ideal for representing functions that depend on directions using just *two coordinates* (the azimuth θ and the inclination ϕ; the radius is fixed to 1). Spherical Harmonics are a family with two parameters:

- Band: $l \in \mathbb{N} \cup \{0\}$
- Mode: $m \in \mathbb{Z} \cap [-l, l]$

Therefore, they are usually referred to as $Y_l^m(\theta, \phi)$, where θ is the azimuth and ϕ is the inclination. The number of coefficients needed to represent a function using Spherical Harmonics up to band L is given by:

$$N = \sum_{l=0}^{L} (2l + 1) = L + 1 + 2 \cdot \frac{L(L+1)}{2} = L + 1 + L(L+1) = (L+1)^2$$

Observación. It should not be confused how many bands are going to be used with up to which band we are going to use.

- Up to band L: we are going to use all the bands from 0 to L, which means that we are going to use $L+1$ bands, and therefore $(L+1)^2$ coefficients.
- L bands: we are going to use the bands from 0 to $L-1$, which means that we are going to use L bands, and therefore L^2 coefficients.

10.2.2. Monte Carlo Integration

The coefficients of the projection of a function f onto the Spherical Harmonics basis have to be computed using an integral over the unit sphere. To compute these integrals, Monte Carlo Integration is usually used. Monte Carlo Integration is a numerical method for approximating integrals using random sampling. The idea is to sample points from the domain of the function and use the average value of the function at those points to approximate the integral. The more samples we take, the better the approximation will be.

- Monte Carlo Integration is particularly useful for high-dimensional integrals, where traditional numerical methods may struggle.
- It is an abstraction of the Riemann sum, where instead of using a regular grid of points, we use random samples.

Therefore, to compute the coefficients of the projection of a function f onto the Spherical Harmonics basis, we can use Monte Carlo Integration and just sample the function at random points on the unit sphere.

10.3. PRT Pipeline

The PRT pipeline consists of two main stages: the precomputation stage and the rendering stage. The idea is that as much as possible should be precomputed in the first stage, so that the rendering stage can be as fast as possible.

10.3.1. Precomputation Stage

In order to be able to solve the Rendering Equation in real-time, we need to precompute the following information (assuming that we use L bands for approximating):

1. The values of the L^2 SH in each direction. These are no integrals, but they are needed to project the other functions.
2. The projection of the term $L_{\text{env}}(\omega')$ onto the SH basis, which gives us L^2 coefficients for each color channel (R, G, B). For each of the sample directions for the Monte Carlo Integration, we need:
    - The value of the environment map in that direction, which can be obtained by sampling the environment map.

- The value of the SH basis functions in that direction, which can be obtained from the precomputed values of the SH basis functions.

3. For each vertex x in the scene, the projection of the transfer function $T(x, \omega')$ onto the SH basis, which gives us L^2 coefficients for each vertex. For each of the sample directions for the Monte Carlo Integration, we need to compute the following terms:

   - The visibility term $V(x, \omega')$ can be computed using ray tracing, which is a computationally expensive process. However, given that this is done in the precomputation stage, it is not a problem. Depending how far the ray collides, this term will have a higher or lower value.

     When just considering the first Van Neumann term, the visibility term can be computed simply by tracing a ray an returning 1 if the ray does not collide with any object, and 0 if it does. However, when considering more Van Neumann terms, the visibility term can take fractional values, as the values are averaged.
   - The term $(\omega' \cdot N)$ can be computed using the normal of the vertex and the direction of the incoming light.
   - The value of the SH basis functions in that direction, which can be obtained from the precomputed values of the SH basis functions.

This way, we have precomputed all the information needed to evaluate the Rendering Equation in real-time. The precomputation stage can be computationally expensive, especially for the third step, where we need to compute the visibility term for each vertex and each sample direction. However, this is done offline, so it does not affect the real-time performance of the rendering stage. In this point, we have:

$$L_{\text{env}}(\omega') \approx \sum_{i=0}^{(L-1)^2} l_i b_i(\omega') \qquad T(x, \omega') \approx \sum_{i=0}^{(L-1)^2} t_{x,i} b_i(\omega')$$

10.3.2. Rendering Stage

In the rendering stage, we can now evaluate the Rendering Equation in real-time using the precomputed information. Given a point x and a direction ω, we can compute the outgoing radiance as:

$$\begin{aligned}
L_o(x, \omega, \lambda) &= \frac{\rho}{\pi} \int_\Omega L_{\text{env}}(\omega') \cdot V(x, \omega') \cdot (\omega' \cdot N)\ d\omega' = \\
&= \frac{\rho}{\pi} \int_\Omega L_{\text{env}}(\omega') \cdot T(x, \omega')\ d\omega' \approx \\
&\approx \frac{\rho}{\pi} \int_\Omega \left(\sum_{i=0}^{(L-1)^2} l_i b_i(\omega') \right) \left(\sum_{j=0}^{(L-1)^2} t_{x,j} b_j(\omega') \right)\ d\omega' \overset{(*)}{=} \\
&\overset{(*)}{=} \frac{\rho}{\pi} \sum_{i=0}^{(L-1)^2} l_i t_{x,i} \int_\Omega b_i(\omega') b_i(\omega')\ d\omega' = \frac{\rho}{\pi} \sum_{i=0}^{(L-1)^2} l_i t_{x,i}
\end{aligned}$$

where in $(*)$ we used the fact that the basis is orthogonal. Therefore, we have approximated the integral of the product of L_{env} and T as a dot product of their coefficients in the SH basis. This allows us to evaluate the Rendering Equation in real-time, as we only need to compute a dot product of two vectors of size L^2 for each color channel.

One important thing to note is that the SH are rotationally invariant. That means that if we rotate the scene (or just rotate the light source), the coefficients of the projection of the functions onto the SH basis will change, but the values of the functions themselves will not change. This allows us to, instead of recomputing all the coefficients when the scene is rotated, we can just rotate the coefficients of the projection of the functions onto the SH basis. This is done using a rotation matrix. Therefore, we can achieve real-time rendering of the scene even when the scene is rotated, without having to recompute all the coefficients. However, PRT can't be used when the objects in the scene are moving, as the visibility term will change, and therefore we would need to recompute all the coefficients for each vertex, which is not feasible in real-time. Therefore, PRT is only suitable for *static scenes*, where the objects are not moving.

11. Ambient Occlusion

Ambient occlusion is a shading method that calculates how much ambient light can reach a point on a surface. It is used to add depth and realism to 3D scenes by simulating the soft shadows that occur in areas where light is partially blocked by nearby geometry. However, it should be noted that ambient occlusion is not a shadowing technique in the traditional sense, as it does not account for direct light sources or cast shadows. Instead, it focuses on the occlusion of ambient light, which is the diffuse light that is scattered in all directions.

The idea that lays behind ambient occlusion is that points on a surface that are close to other geometry will receive less ambient light and therefore appear darker. This is because the nearby geometry blocks some of the ambient light from reaching those points. Everything is calculated based on the Ambient Occlusion Factor (Equation 11.1), which is a value between 0 and 1 that represents the amount of ambient light that can reach a point on a surface. A value of 0 means that the point is completely occluded and receives no ambient light, while a value of 1 means that the point is fully exposed and receives all ambient light.

$$k_a(p) = \frac{1}{\pi} \int_{\Omega^+} V\left(\overrightarrow{\omega}\right) \cos\theta \; d\overrightarrow{\omega} \tag{11.1}$$

where:

- Ω^+ is the normal-oriented hemisphere above the point p.
- $V\left(\overrightarrow{\omega}\right)$ is the visibility function, which is 1 if the direction $\overrightarrow{\omega}$ is not occluded by any geometry and 0 if it is occluded. This is calculated by casting a ray from the point p in the direction $\overrightarrow{\omega}$ and checking for intersections with other geometry in the scene.
- $\cos\theta$ is the cosine of the angle between the normal at point p and the direction $\overrightarrow{\omega}$. This term accounts for the fact that light arriving at a grazing angle contributes less to the ambient illumination than light arriving directly.
- The factor of $1/\pi$ is used to normalize the result, because the integral of the cosine term over the hemisphere is equal to π; this ensures that the ambient occlusion factor is properly scaled to the range $[0, 1]$.

Therefore, AO is described as follows: "Trace rays through the normal-oriented unit hemisphere, and return the cosine-weighted percentage of rays that do not hit any geometry." This method is computationally expensive, as it requires casting many rays for each point on the surface to accurately estimate the ambient occlusion factor. However, there are various techniques to approximate ambient occlusion.

11.1. Raytracing AO

The idea behind raytracing AO is to preprocess everything for the static geometries in the scene, and store the results in a occlusion map, which essentially is a texture. This method is very efficient for static objects, as the ambient occlusion can be calculated once and reused for each frame. However, it does not help with dynamic objects.

11.2. Object Space AO

Object space AO is a method that calculates ambient occlusion in the object space of a 3D model. This technique represents each vertex of the model as a circle with the same normal as the vectex and with an area equal to one third of the the sum of the areas of the triangles that share the vertex[1]. This representation can be fastly calculated, as the lenght of the sides of the triangles are just an square root, and then there are just some simple calculations to find the area of the triangle known the length of its sides:

- Half of the module of the cross product of the two edges of the triangle (the vectors are needed).
- Heron's formula (11.2). If the lengths of the sides of the triangle are a, b, and c, then:

$$A = \sqrt{s(s-a)(s-b)(s-c)} \qquad \text{where } s = \frac{a+b+c}{2} \tag{11.2}$$

Once the circles are created, the key aspect of this technique is that, in order to calculate the AO factor for a vertex, no ray casting is needed. Instead, the integrand of Equation (11.1) is approximated by the Equation (11.3), which represent's the accessibility of the vertex E (emitter point) from the vertex R (receiver point):

$$\text{accessibility}(R,E) = 1 - \frac{\text{máx}(0,\cos(\theta_E)) \cdot \text{máx}(0,4\cos(\theta_R))}{\sqrt{\frac{A_R}{\pi} + r^2}} \tag{11.3}$$

The Figure 11.1 represents the terms used in the Equation (11.3). Therefore, the Equation (11.4) is the final formula to calculate the AO factor for a vertex E using the object space AO method, where N is the number of vertices in the model:

$$k_a(E) = \frac{1}{N}\sum_{R=1}^{N} \text{accessibility}(R,E) \tag{11.4}$$

This method is really efficient, as no ray casting is needed. In addition, in order to increase the performance, the accessibility can be stored and only recalculated for the vertices that have changed their position or normal. This way, it also works for dynamic objects.

[1] That way, the area of each triangle is distributed equally among its three vertices.

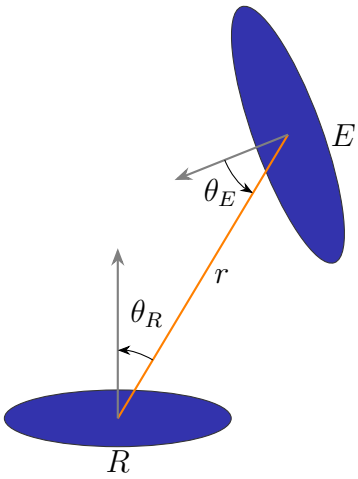


Figura 11.1: Object Space AO: Schematic representation of the accessibility of the vertex E (emitter point) from the vertex R (receiver point).

Shadowed Occluders

This approach faces a problem with the already shadowed occluders, which are the occluders that are themselves in shadow. For example: a point on the surface may have an object nearby that blocks light, but if that object is in turn covered by another geometry, we should not add both shadows. If we did, the point would appear artificially dark. Only the main occlusion should determine the final lighting.

In order to solve this problem, the shadowed occluders should have less influence on the final AO factor. Multiple passes can be used to calculate the AO factor, and in each pass, the AO factor should be updated by multiplying the previous AO factor with the new one calculated in the current pass. This way, the influence of shadowed occluders is reduced, as their contribution to the AO factor will be multiplied by a value less than 1 in subsequent passes.

11.3. Bent Normals

Bent normals are a technique used in ambient occlusion to account for the directionality of light. Instead of just calculating the amount of ambient light that reaches a point on a surface, the normal shading is applied. However, the usual normal is not used, but a bent normal is calculated, which points in the direction of the most unoccluded path from the point on the surface. This allows for more accurate shading, as it takes into account the fact that light may be coming from a specific direction, rather than being uniformly distributed.

This technique obviously faces problems where more than one path is unoccluded, as it only considers the most unoccluded path. Therefore, it does not always provide the most accurate results, especially in complex scenes with multiple light sources and occluders.

11.4. Screen Space AO

Screen Space Ambient Occlusion (SSAO) is a real-time technique used in computer graphics to approximate ambient occlusion effects. It operates in screen space, meaning it firstly renders the scene to a depth buffer and then uses that information to calculate the ambient occlusion for each pixel on the screen[2]. This makes this technique more efficient, as only what is visible on the screen is processed, rather than the entire scene.

For each pixel, SSAO samples the surrounding pixels (creating an sphere around the pixel) and checks the depth values to determine how much of the surrounding geometry is in front of the current pixel, using therefore the z-buffer to recreate the profile. The more geometry that is in front, the higher the occlusion factor, which results in a darker pixel. The reader may think that this method introduces a lot of noise, as in the edges between the occluded and non-occluded areas weird patterns may appear. However, this technique produces really good results, and is widely used in real-time applications given that it only processes what is visible on the screen, which makes it much faster than other methods that require processing the entire scene.

11.4.1. Deferred Shading

As mentioned in the previous section, SSAO usually uses deferred shading, which is a rendering technique that separates the rendering process into two main stages: geometry pass and lighting pass.

1. In the geometry pass, the scene is rendered and each of the necessary attributes (such as normals, depth, color, etc.) are stored in different textures, collectively known as the G-buffer[3]. This allows for the storage of all the information needed for lighting calculations without having to re-render the geometry for each light source.

2. In the lighting pass, only the visible pixels are processed. The whole image is covered using just two triangles, and for each pixel, the information stored in the G-buffer is used to calculate the lighting, including the ambient occlusion using SSAO. This way, the lighting calculations are performed only on the pixels that are visible on the screen, which significantly improves performance compared to traditional forward rendering techniques where lighting is calculated for every object in the scene regardless of its visibility.

11.4.2. SSAO Extensions

There are various extensions to the basic SSAO technique that aim to improve the quality of the results or to add additional features. Some of them also include using the normals stored in the G-buffer to take into account the directionality of light, which can help to reduce the noise and improve the overall appearance of the ambient occlusion.

[2]This is achieved using Deferred Shading, see Section 11.4.1.
[3]Geometry Buffer, as they store information about the geometry.

Horizon-Based Ambient Occlusion (HBAO)

Horizon-Based Ambient Occlusion (HBAO) is an extension of the SSAO technique that aims to provide higher quality results. For every pixel, HBAO also uses an integral approach to calculate the ambient occlusion factor, but instead of integrating over the hemisphere, it integrates over the circle that encloses the pixel. For each direction θ in the circle, calculates:

- The tangent angle $T(\theta)$: the angle between the tangent vector (to the surface at pixel p) and the x axis.
- The horizon angle $H(\theta)$: the angle between the horizon vector and the x axis. The horizon vector is the vector that points to the first occluder in a given direction, which is calculated by sampling the depth buffer along that direction until an occluder is found.

Therefore, the ambient occlusion factor in the pixel p is:

$$AO(p) = 1 - \int_{-\pi}^{\pi} (\sin(H(\theta)) - \sin(T(\theta)))\, W(\theta) d\theta$$

where $W(\theta)$ is just a factor needed so that $AO \in [0, 1]$. Even though it is slower because of the integral, this method is still really efficient because it also uses Deferred Shading. It provides better results than basic SSAO, as it takes into account the geometry of the scene more accurately, but it is also more computationally expensive.

This method also faces problems at the edges of the screen, as the horizon vector may not be properly calculated due to the lack of surrounding pixels. In order to solve this problem, the image is usually rendered in a quite larger scene, and then the edges are cropped to fit the screen. This way, the horizon vector can be calculated properly even at the edges of the screen, which improves the overall quality of the ambient occlusion effect.